

Index

Sr. No	Topic	Pg.no
1	Design and implement basics embedded circuits 1. Automatic Alarm system- Alarm should get triggered by sensor 2. Timer based buzzer 3. Sensor based Counting device. <u>(PROTEUS)</u>	3 - 24
2	Demonstrate communication between two embedded devices using UART port. <u>(PROTEUS)</u>	25 - 29
3	Built an IoT system to send ticket before entering the bus. <u>(PROTEUS)</u>	30 - 36
4	Develop a IoT application which will record the movement and orientation of your phone and give the data back to the PC. <u>(MIT inventor)</u>	37 - 39
5	Develop an IoT application that will raise an alarm whenever with going to rain outside based on the weather prediction data. <u>(CISCO)</u>	40 - 42
6	Deploy an IoT application which will alert you by beeping or vibrating your phone whenever you get someone call your name. <u>(MIT inventor)</u>	43 - 46
7	Develop an IoT application for monitoring water levels in tanks and automatically start the motor to fill the tank if the level goes below the critical level. <u>(PROTEUS)</u>	47 - 50
8	Develop an IoT module to which measure the intensity of light and send the same to your PC/ Phone. <u>(MIT inventor)</u>	51 - 52

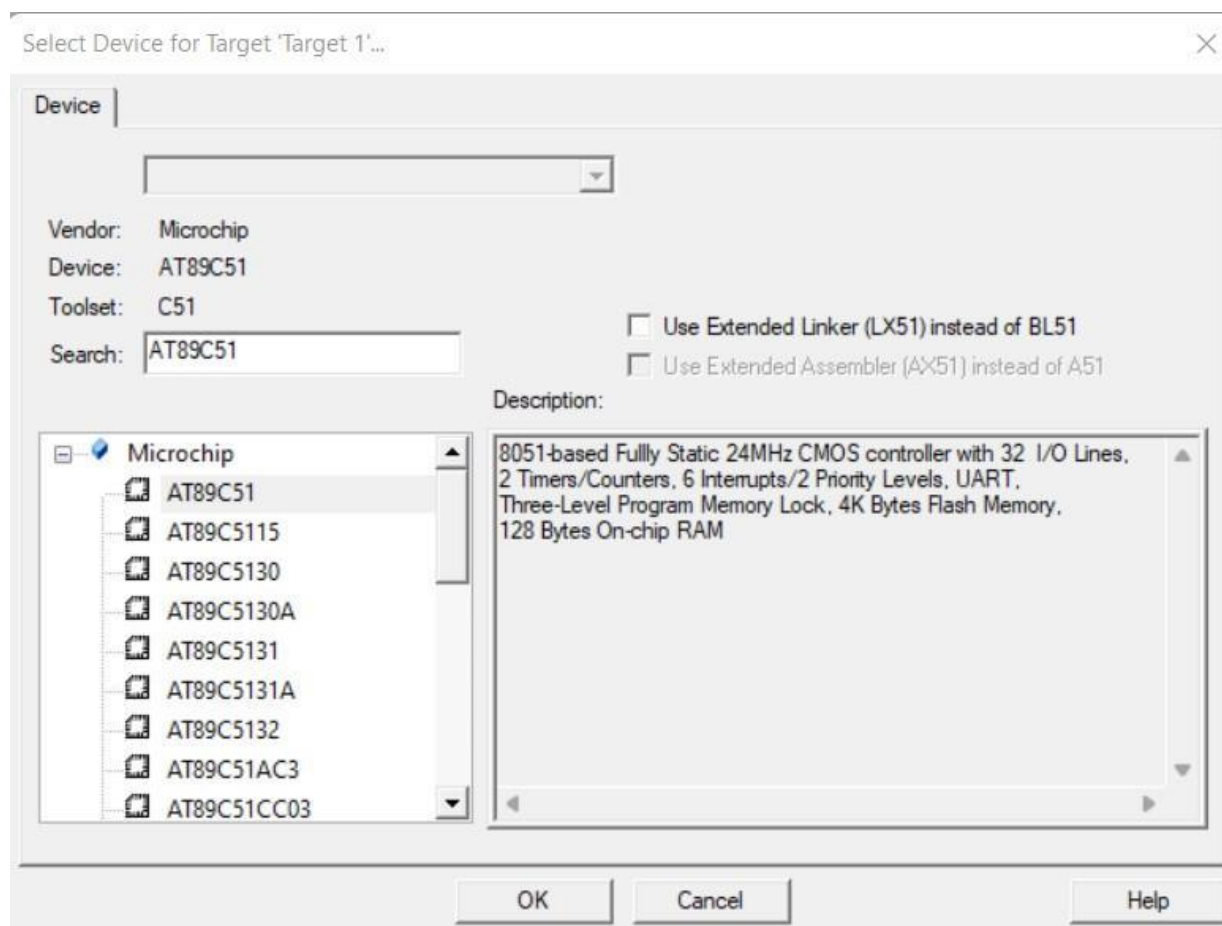
9	Develop an IoT application for Motion detection. <u>(CISCO)</u>	53 - 55
---	---	---------

Practical: 1.1

Automatic Alarm system- Alarm should get triggered by sensor

Part 1: Writing C Program for alarm system in Keil Software and creating Hex file.

Step 1: Project → New Project → Search your Folder → Give name to project → Select Atmel → AT89C51 → Click Ok.



Step 2: File ? New file ? Text Window is open ? Type program ? Save file by .c extension

Code:

```
#include<reg51.h>
sbit sensor=P1^0;
sbit rs=P1^1;
sbit rw=P1^2;
sbit en=P1^3;
sbit buzzer=P3^0;
void lcdinit();
void lcddat(unsigned char);
void lcdcmd(unsigned char);
void lcddis(unsigned char *);
void delay();
void main()
{
    buzzer=0;
    sensor=0;
    back:lcdinit();
    lcddis("FIRE ALERT");
    lcdcmd(0xc0);
    lcddis("SYSTEM");
    delay();
    while(1)
    {
        if(sensor==1)
        {
            lcdcmd(0x01);
            lcddis("FIRE OCCURED");
            buzzer=1;
        }
        goto back;
    }
}
```

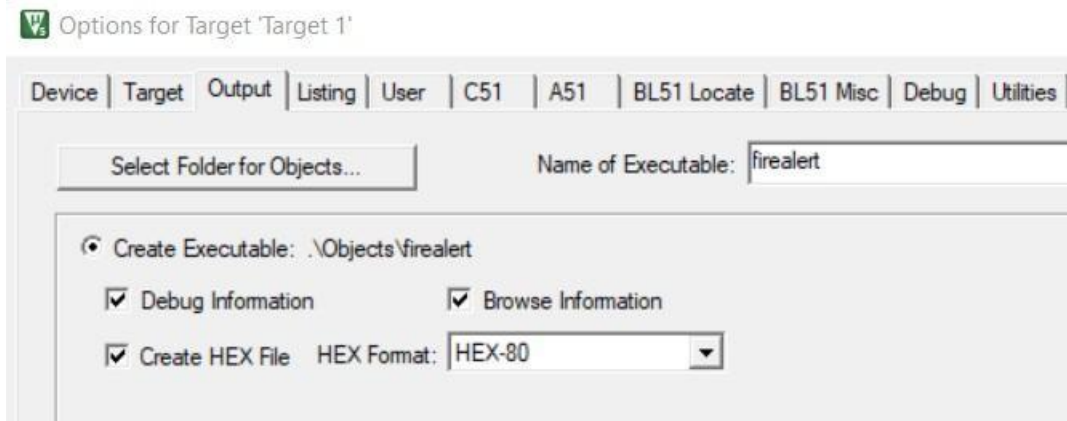
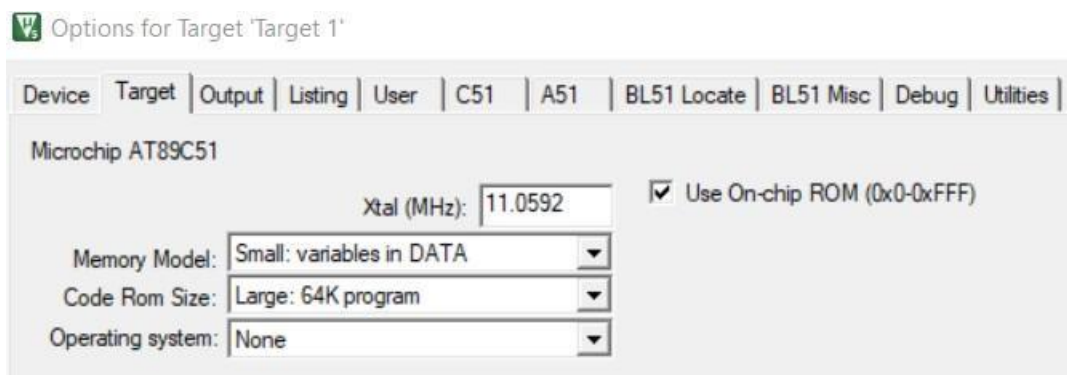
```

}
void lcdinit()
{
    lcdcmd(0x38);
    lcdcmd(0x01);
    lcdcmd(0x10);
    lcdcmd(0x0c);
    lcdcmd(0x80);
}
void lcdcmd(unsigned char val)
{
    P2=val;
    rs=0;
    rw=0;
    en=1;
    delay();
    en=0;
}
void lcddat(unsigned char dat)
{
    P2=dat;
    rs=1;
    rw=0;
    en=1;
    delay();
    en=0;
}
void lcddis(unsigned char *s)
{
    unsigned char w;
    for(w=0;s[w]!='\0';w++)
    {
        lcddat(s[w]);
    }
}
void delay()
{
    unsigned int v1;
    for(v1=0;v1<10000;v1++);
}

```

Step 3: Click on target ? Source group ? right click ? Add existing file ? select program file ? Click on Add.

Step 4: Right click on target ? Option for target ? Change frequency to 11.0592 MHz ? Click on use on-chip ROM ? Go to output label ? Click on Create Hex file ? Ok.



Step 5: Project ? Click on Build target.

```

Build Output
Rebuild started: Project: firealert
Rebuild target 'Target 1'
compiling firealert.c...
linking...
Program Size: data=13.0 xdata=0 code=248
creating hex file from ".\Objects\firealert"...
".\Objects\firealert" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:14

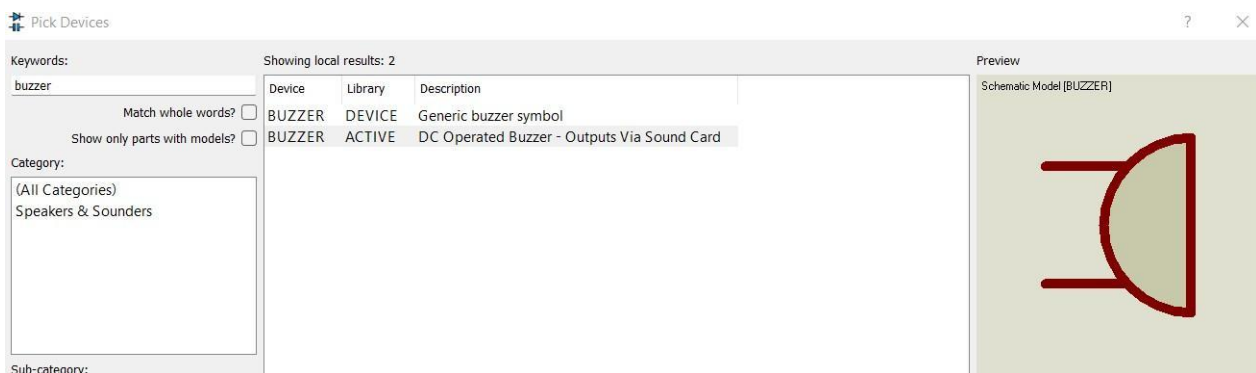
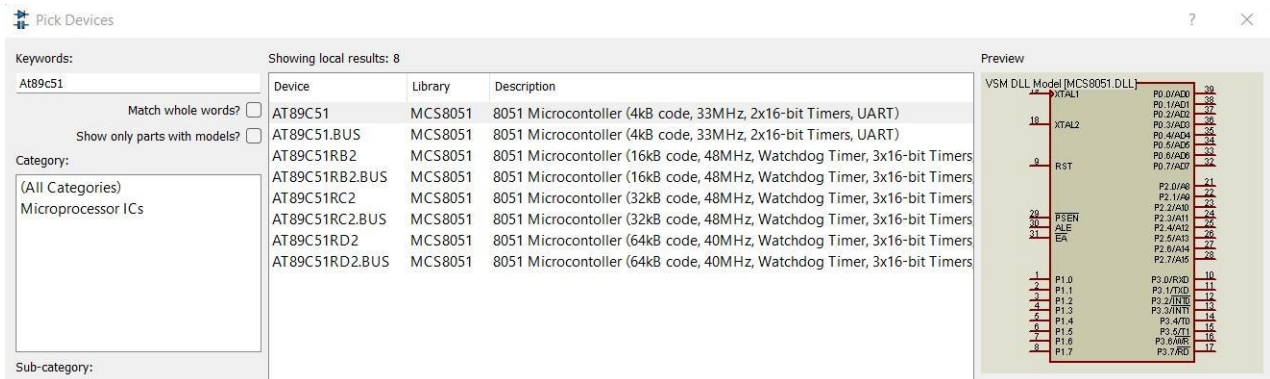
```

Part 2: Designing circuit for Alarm system in proteus.

Step 1: Proteus 8 professional ? New project ? name for project ? Finish.

Step 2: From P (Device selector) search and add the components

- ☐ AT89C51
- ☐ Buzzer active DC Operated Buzzer Output Via Sound Card
- ☐ LCD: LM016L 16x2 Alphanumeric LCD
- ☐ Button (SPST Push button)



Pick Devices

Keywords: Showing local results: 108

Match whole words? ☐ Show only parts with models? ☐

Category:

- (All Categories)
- Analog ICs
- CMOS 4000 series
- Microprocessor ICs
- Optoelectronics
- TTL 74HC series

Sub-category:

Device	Library	Description
LCDSTK502	DISPLAY	Atmel STK-502 Custom LCD Display
LCDSTK504	DISPLAY	Atmel STK-504 Custom LCD Display
LGM12641BS1R	DISPLAY	128x64 Graphical LCD with KS0108 controllers
LM016L	DISPLAY	16x2 Alphanumeric LCD
LM017L	DISPLAY	32x2 Alphanumeric LCD
LM018L	DISPLAY	40x2 Alphanumeric LCD
LM020L	DISPLAY	16x1 Alphanumeric LCD
LM032L	DISPLAY	20x2 Alphanumeric LCD
LM041L	DISPLAY	16x4 Alphanumeric LCD
LM044L	DISPLAY	20x4 Alphanumeric LCD
LM3228	DISPLAY	128x64 Graphical LCD
LM3229	DISPLAY	240x128 Graphical LCD
LM3267	DISPLAY	240x64 Graphical LCD
LM3283	DISPLAY	240x64 Graphical LCD

Preview

VSM DLL Model [LCDALPHA]

PCB Preview

Pick Devices

Keywords: Showing local results: 5

Match whole words? ☐ Show only parts with models? ☐

Category:

- (All Categories)
- Microprocessor ICs
- Switches & Relays

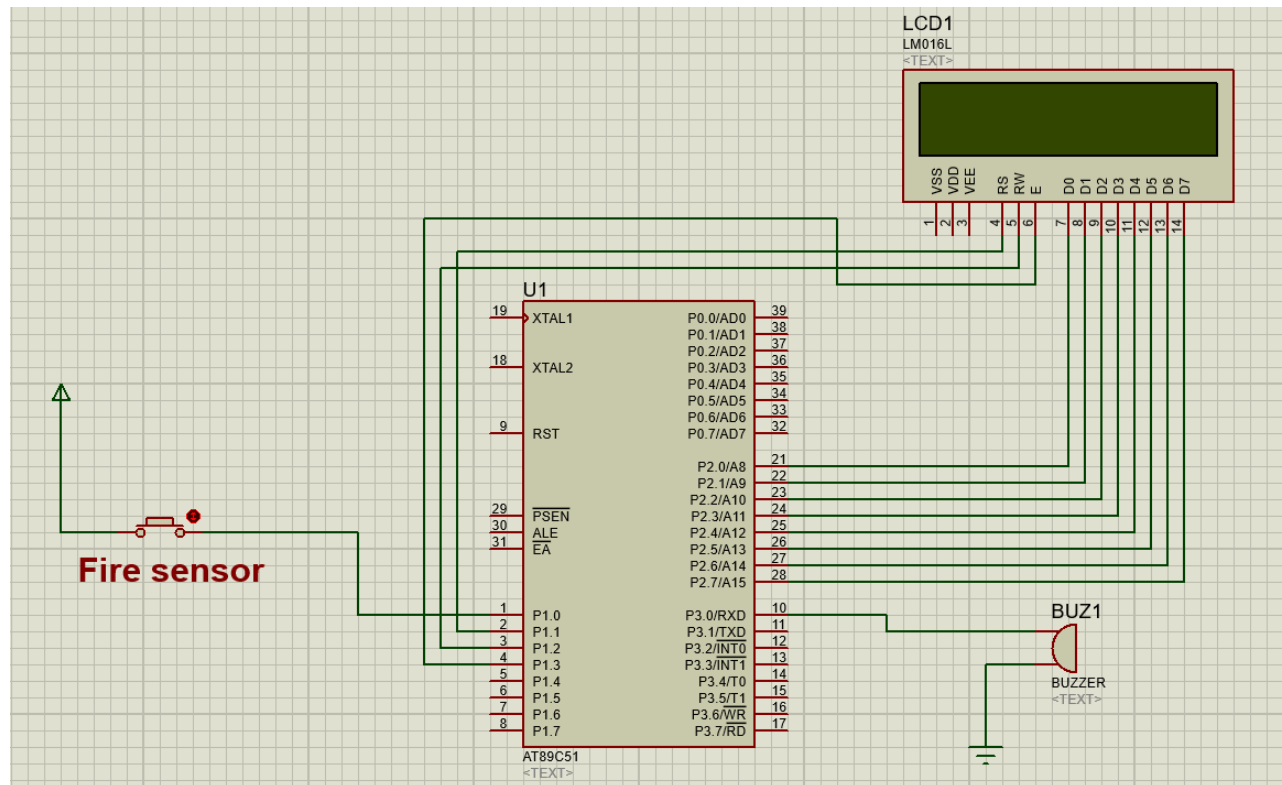
Sub-category:

Device	Library	Description
BUTTON	ACTIVE	SPST Push Button
DS1802	MAXIM	Dual Audio Taper Potentiometer With Pushbutton Control
DS1802E	MAXIM	Dual Audio Taper Potentiometer With Pushbutton Control
DS1802S	MAXIM	Dual Audio Taper Potentiometer With Pushbutton Control
DS1990	MAXIM	Serial Number iButton

Preview

Analogue Primitive [RTSWITCH]

Step 3: Design the Circuit ? Connect ground and power from terminals.



Step 4: Double click on Micro controller ? Select HEX file form program file ? Change Clock frequency to 11.0592MHz ? Click Ok

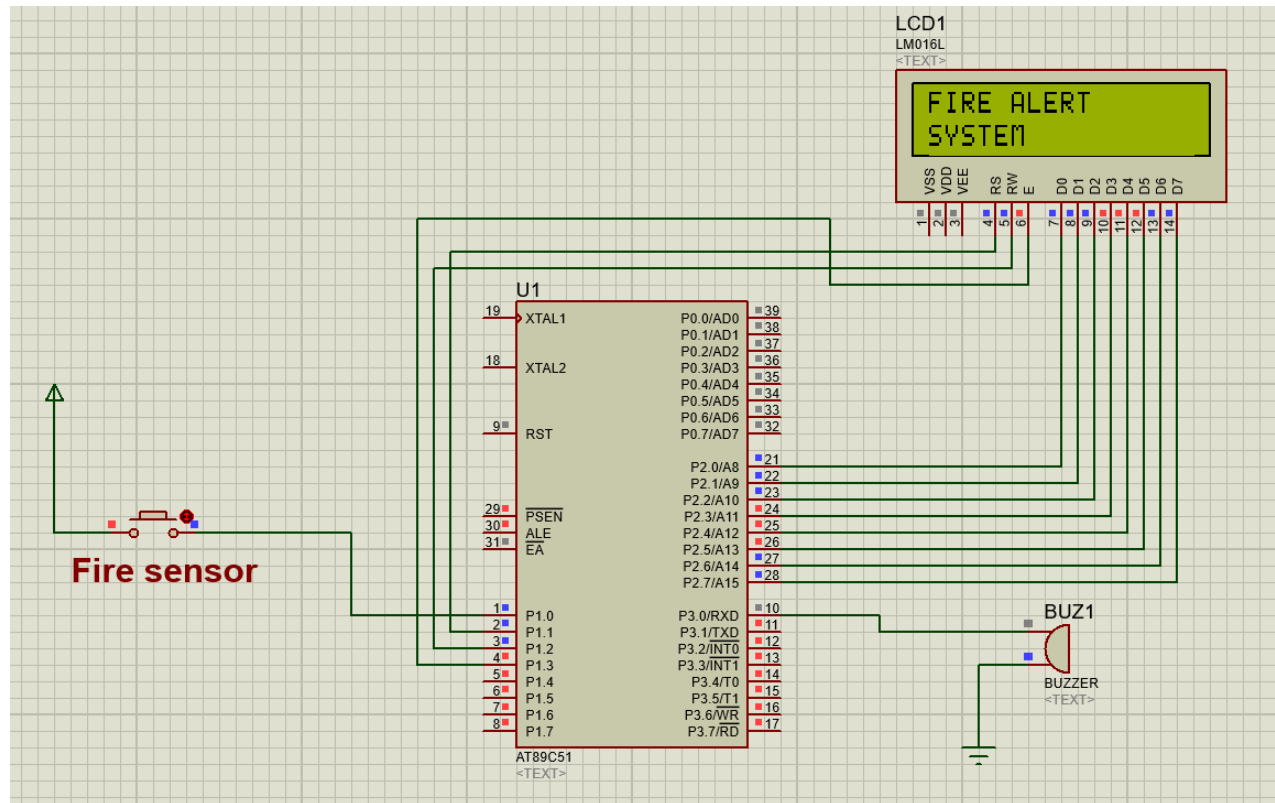
Edit Component

Part Reference:	U1	Hidden:	☐
Part Value:	AT89C51	Hidden:	☐
Element	New		
PCB Package:	DIL40	Hide All	☐
Program File:	D:\M.Sc Sem 2\IOT\Objects\fireale	Hide All	☐
Clock Frequency:	11.0592MHz	Hide All	☐
Advanced Properties:			
Enable trace logging	No	Hide All	☐
Other Properties:			

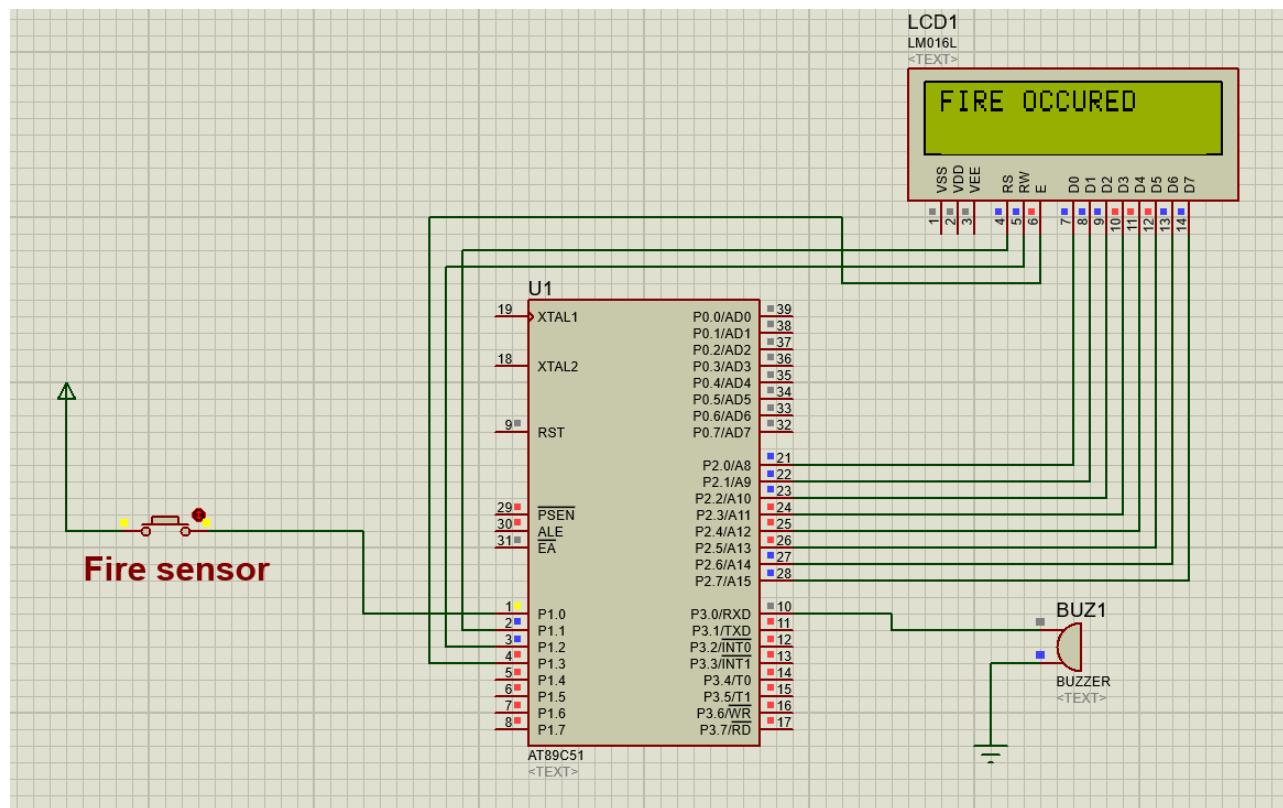
Buttons: OK, Help, Data, Hidden Pins, Edit Firmware, Cancel

Output: Start the simulation.

□ Here when Fire sensor is OFF i.e., fire not detected.



□ When fire is detected i.e., Fire sensor in ON. LCD is alerting that FIRE OCCURED.

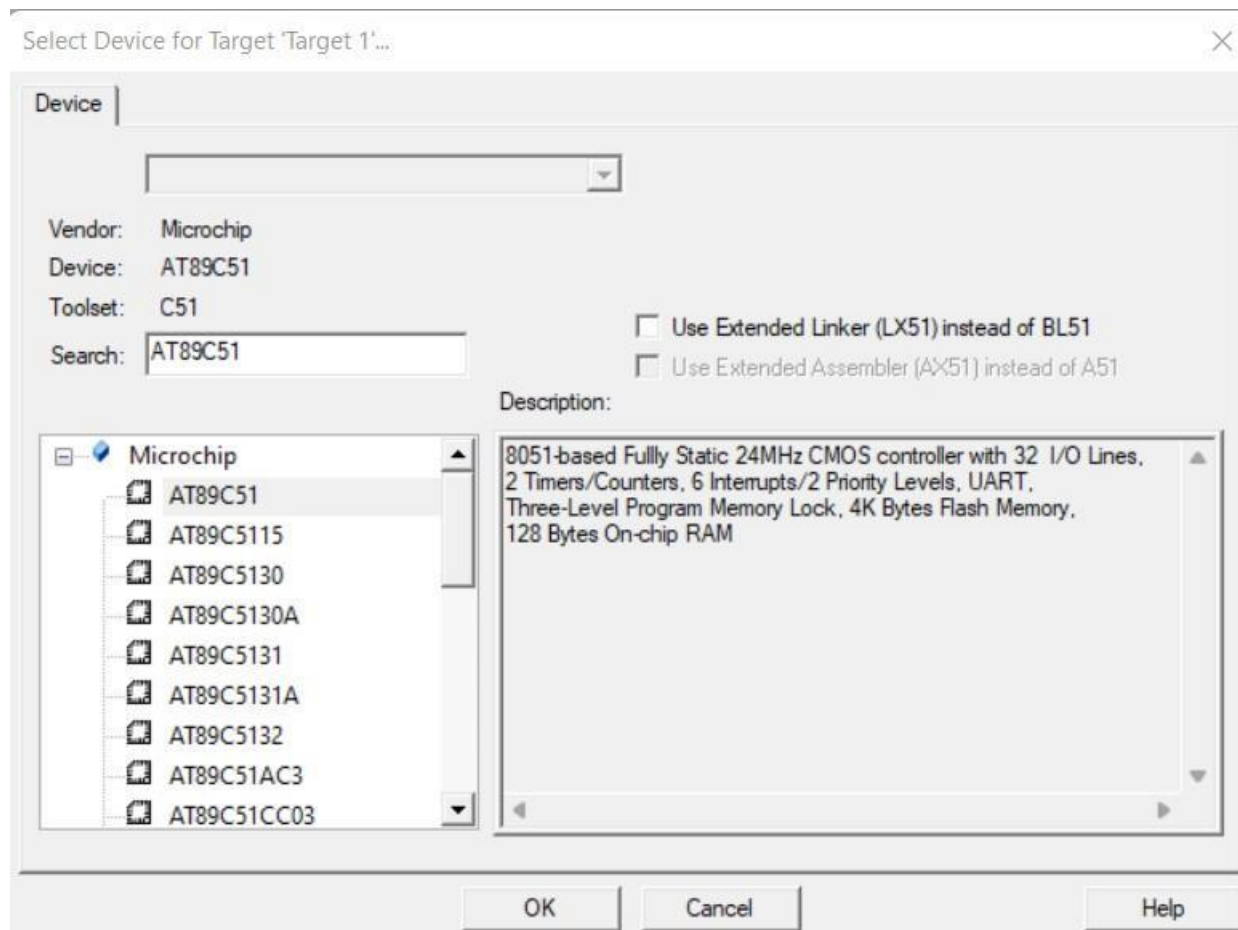


Practical: 1.2

Timer based buzzer

Part 1: Writing C Program for Timer based buzzer in Keil Software and creating Hex file.

Step 1: Project → New Project → Search your Folder → Give name to project → Select Atmel → AT89C51 → Click Ok.



Step 2: File → New file → Text Window is open → Type program → Save file by .c extension

Code:

```
#include<REG51.h>
#include<stdio.h>
//buzzer
sbit buzzer1=P3^0;
void delay(int count);
void main()
{
    P3=0x00;
    while(1)
    {
        delay(100);
        buzzer1=1;
    }
}
void delay(int count)
{
    int i,j;
    for (i=0;i<count;i++);
    for (j=0;j<113;j++);
}
```

Step 3: Click on target ? Source group ? right click ? Add existing file ? select program file ? Click on Add.

Step 4: Right click on target ? Option for target ? Change frequency to 11.0592 MHz ? Click on use on-chip ROM ? Go to output label ? Click on Create Hex file ? Ok.

 Options for Target: 'Target 1'

Device	Target	Output	Listing	User	C51	A51	BL51 Locate	BL51 Misc	Debug	Utilities
Microchip AT89C51										
						Xtal (MHz):	11.0592	☒ Use On-chip ROM (0x0-0xFFFF)		
Memory Model:		Small: variables in DATA								
Code Rom Size:		Large: 64K program								
Operating system:		None								

Step 5: Project ? Click on Build target.

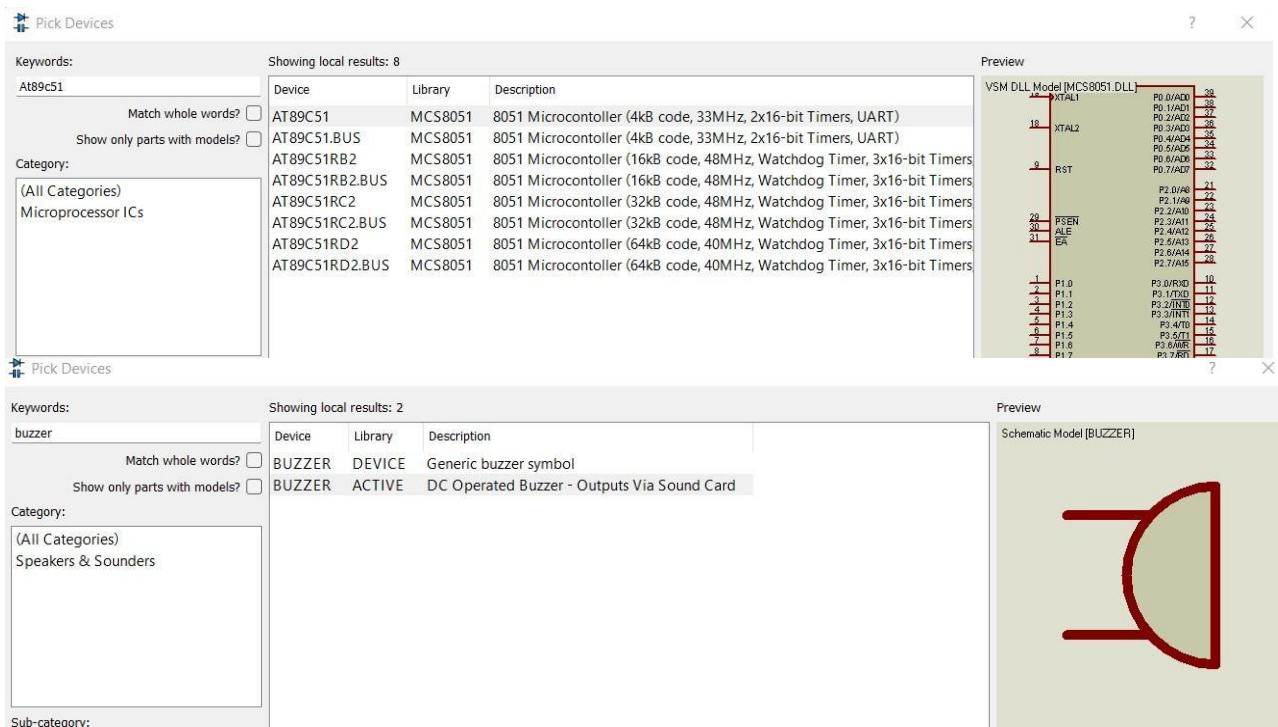
```
Build Output
Rebuild started: Project: buzzer
Rebuild target 'Target 1'
compiling buzzer.c...
linking...
Program Size: data=9.0 xdata=0 code=67
creating hex file from ".\Objects\buzzer"...
".\Objects\buzzer" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:00
```

Part 2: Designing circuit for timer based buzzer in proteus.

Step 1: Proteus 8 professional ? New project ? name for project ? Finish.

Step 2: From P (Device selector) search and add the components

- ☐ AT89C51
- ☐ Buzzer active DC Operated Buzzer Output Via Sound Card
- ☐ Speaker- loudspeaker model(analog)- output via sound card
- ☐ Potentiometer- POT-HG
- ☐ Low-frequency low-power silicon NPN Transistor
- ☐ DC voltmeter from instruments.



Pick Devices

Keywords: speaker

Showing local results: 6

Device	Library	Description
ARDUINO_AUDIO	ACTIVE	Audio Element of Arduino WAVE Shield based on MCP4921 DAC.
BUZZER	DEVICE	Generic buzzer symbol
BUZZER	ACTIVE	DC Operated Buzzer - Outputs Via Sound Card
SOUNDER	ACTIVE	Piezo Sounder model (Digital) - Outputs Via Sound Card
SPEAKER	DEVICE	Generic loudspeaker symbol
SPEAKER	ACTIVE	Loudspeaker model (Analog) - Outputs Via Sound Card

Category: (All Categories) Speakers & Sounders

Sub-category:

Preview: Analogue Primitive [RTVPROBE]

Pick Devices

Keywords: pot-hg

Showing local results: 1

Device	Library	Description
POT-HG	ACTIVE	High Granularity Interactive Potentiometer (Lin, Log or Antilog Law)

Category: (All Categories) Resistors

Preview: VSM DLL Model [HG.POT.DLL]

Pick Devices

Keywords: BC547BP

Showing local results: 1

Device	Library	Description
BC547BP	ZETEX	Low-Frequency Low-Power Silicon NPN Transistor

Category: (All Categories) Transistors

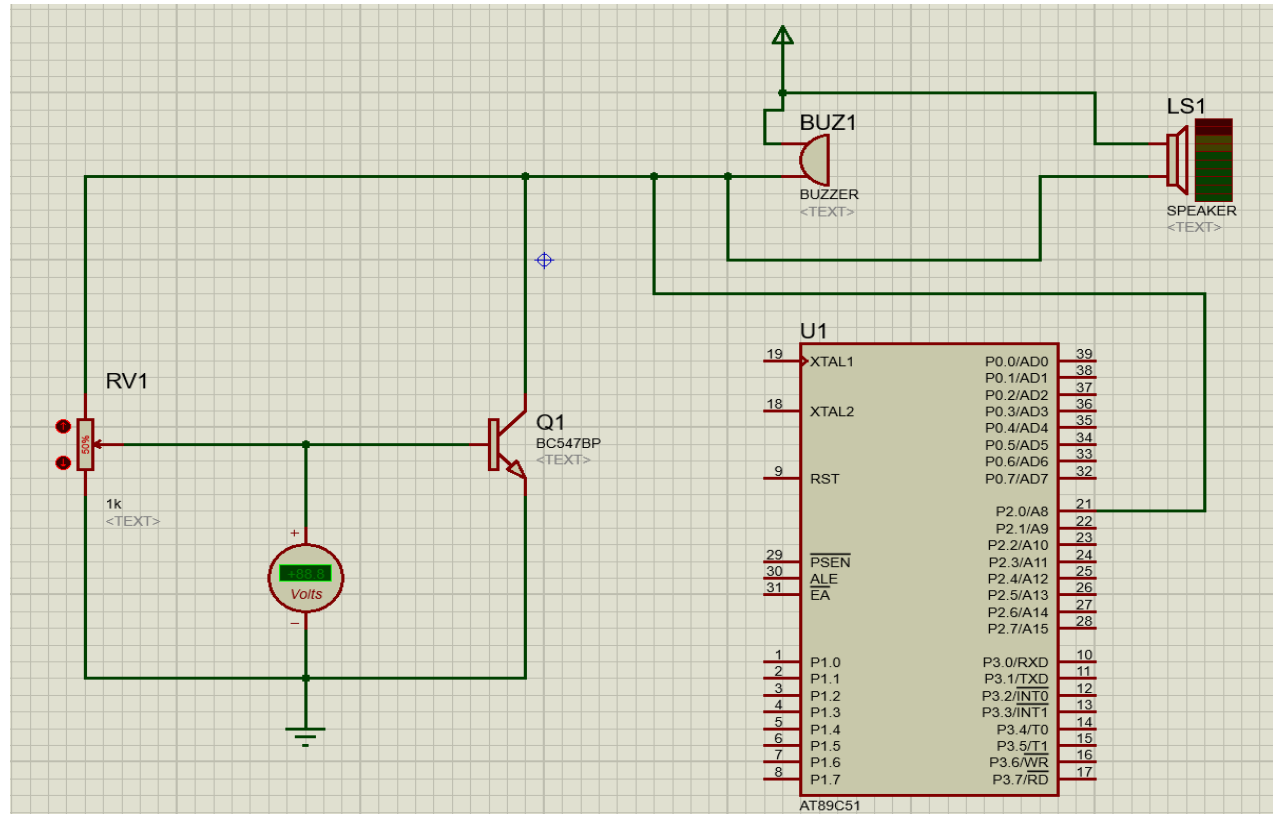
Preview: SPICE Model [BC547BP]

Schematic Capture x PCB

INSTRUMENTS

- OSCILLOSCOPE
- LOGIC ANALYSER
- COUNTER TIMER
- VIRTUAL TERMINAL
- SPI DEBUGGER
- I2C DEBUGGER
- SIGNAL GENERATOR
- PATTERN GENERATOR
- DCVOLTMETER**
- DC AMMETER
- ACVOLTMETER
- AC AMMETER

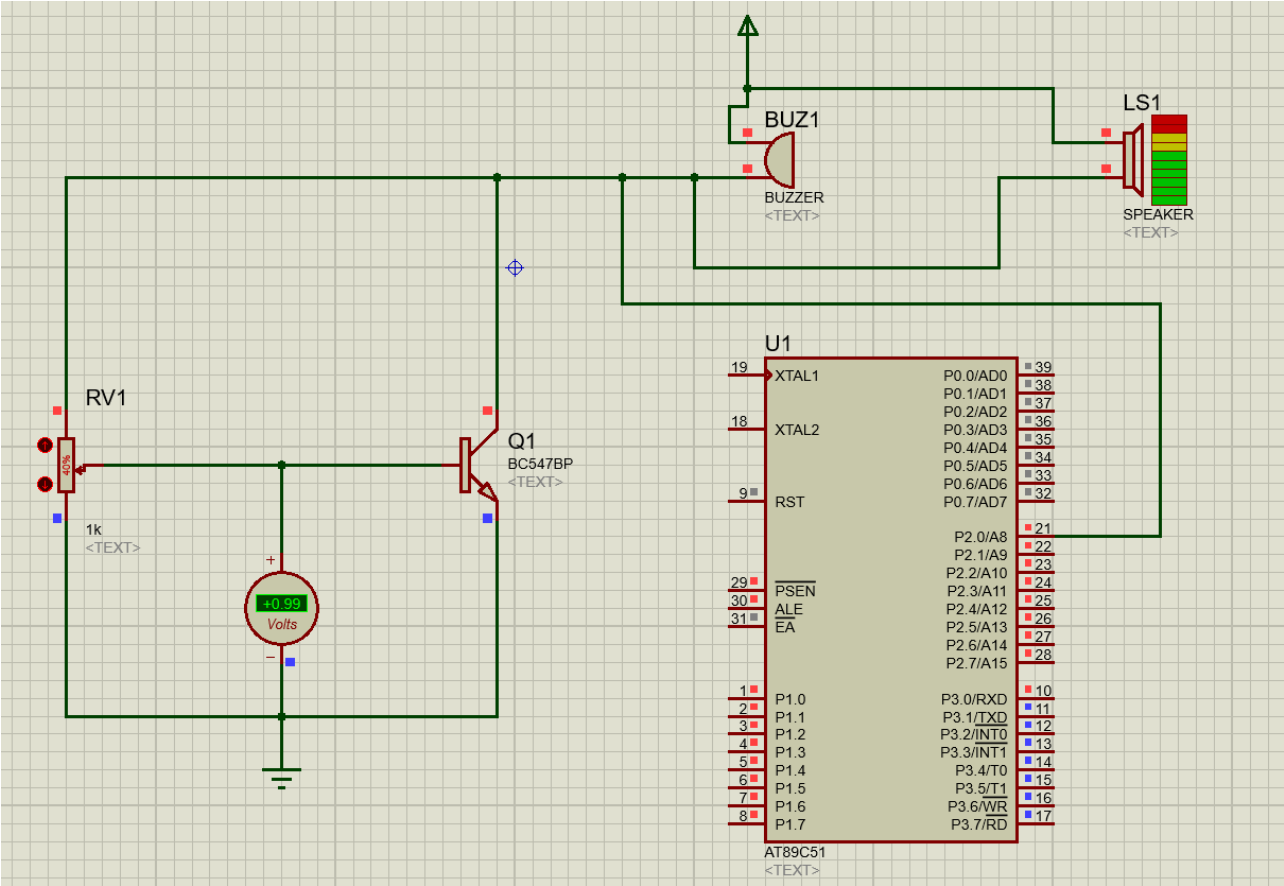
Step 3: Design the Circuit.



Step 4: Double click on Micro controller ? Select HEX file form program file ? Change Clock frequency to 11.0592MHz ? Click Ok

Part Reference:	U1	Hidden:	☐	OK
Part Value:	AT89C51	Hidden:	☐	Help
Element:	New			Data
PCB Package:	DIL40	Hide All	☐	Hidden Pins
Program File:	Objects\buzzer.hex	Hide All	☐	Edit Firmware
Clock Frequency:	11.0592MHz	Hide All	☐	Cancel
Advanced Properties:				

Output: Start the simulation.

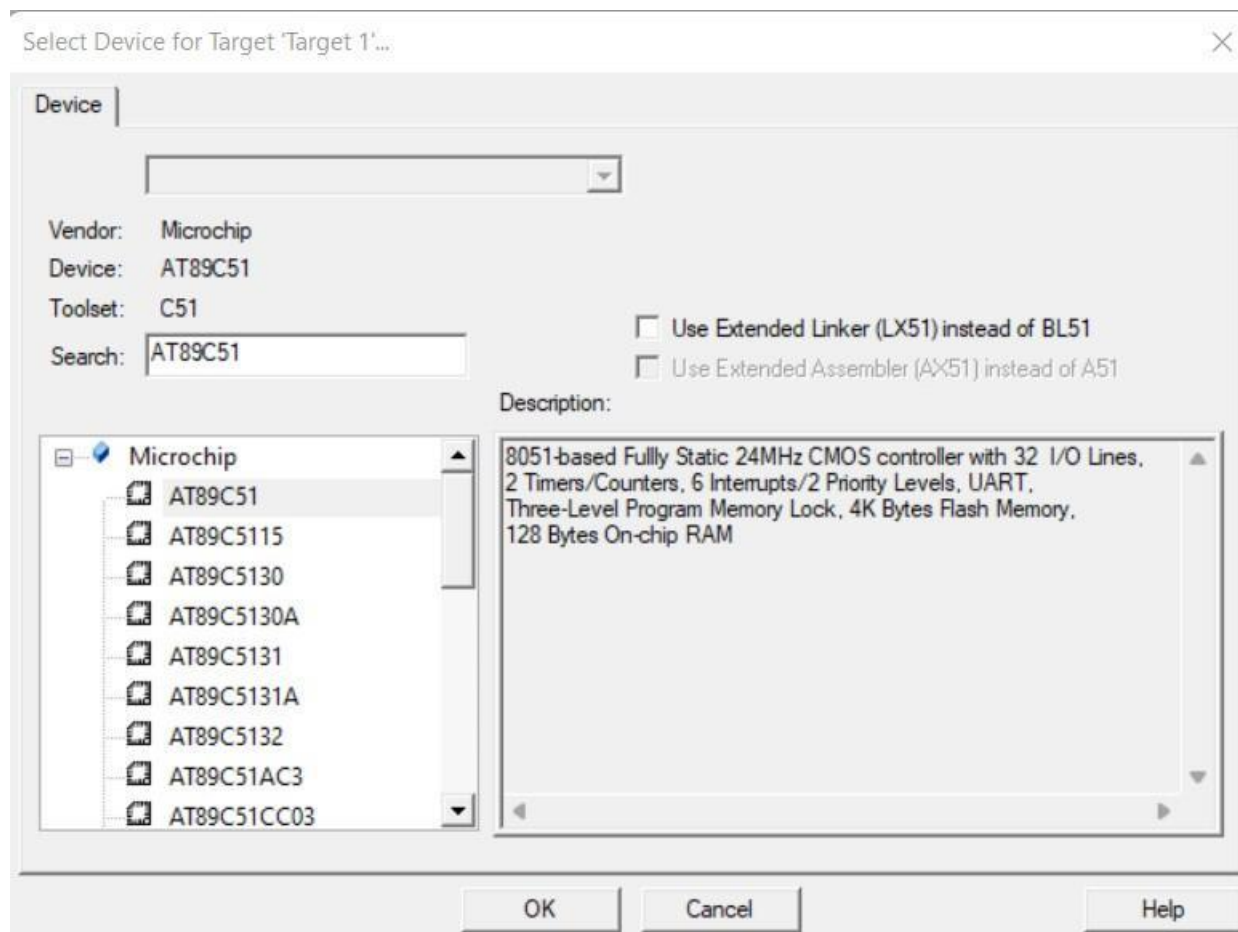


Practical: 1.3

Sensor based Counting device

Part 1: Writing C Program for Sensor based counting device in Keil Software and creating Hex file.

Step 1: Project → New Project → Search your Folder → Give name to project → Select Atmel → AT89C51 → Click Ok.



Step 2: File → New file → Text Window is open → Type program → Save file by .c extension

Code:

```

#include<reg51.h>
sbit sen1=P1^0;    //entry sbit sen2=P1^1; //exit sbit rs=P1^2;
sbit rw=P1^3;
sbit en=P1^4;
void lcdcmd(unsigned char);
void lcddat(unsigned char);
void delay();
void lcddis(unsigned char *s,unsigned char r);
void lcdconv(unsigned char);
void main()
{
    unsigned char x,y;
    lcdcmd(0x38);
    delay();
    lcdcmd(0x01);
    delay();
    lcdcmd(0x10);
    delay();
    lcdcmd(0x0c);
    delay();
    lcddis("WELCOME",7);
    lcdcmd(0xc0);
    delay();
    lcddis("VISITOR COUNTER",15);
    delay();
    lcdcmd(0x01);
    delay();
    while(1)
    {
        if(sen1==0)
        {
            lcdcmd(0x80);
            delay();
            lcddis("ENTRY:",6);
            lcdcmd(0x87);
            delay();
            x=x+1;
            lcdconv(x);
            delay();
        }
    }
}

```

```

        if(sen2==0)
        {
            lcdcmd(0xc0);
            delay();
            lcddis("EXIT:",5);
            lcdcmd(0xc6);
            delay();
            y=y+1;
            lcdconv(y);
            delay();
        }
    }
}

```

```

void lcdcmd(unsigned char val)
{
    P2=val;
    rs=0;
    rw=0;
    en=1;
    delay();
    en=0;
}

```

```

void lcddat(unsigned char dat)
{
    P2=dat;
    rs=1;
    rw=0;
    en=1;
    delay();
    en=0;
}

```

```

void lcddis(unsigned char*s,unsigned char r)
{
    unsigned char w;
    for(w=0;w<r;w++)
    {
        lcddat(s[w]);
        delay();
    }
}

```

```

    }
}

void lcdconv(unsigned char num)
{
    unsigned char p,n;
    p=num/10;
    n=num%10;
    p=num+0x30;
    n=n+0x30;
    lcddat(p);
    lcddat(n);
}

void delay()
{
    unsigned int k,l;
    for(k=0;k<1000;k++);
    for(l=0;l<100;l++);
}

```

Step 3: Click on target ? Source group ? right click ? Add existing file ? select program file ? Click on Add.

Step 4: Right click on target ? Option for target ? Change frequency to 11.0592 MHz ? Click on use on-chip ROM ? Go to output label ? Click on Create Hex file ? Ok.

Options for Target: 'Target 1'

Device	Target	Output	Listing	User	C51	A51	BL51 Locate	BL51 Misc	Debug	Utilities	
Microchip AT89C51											
Xtal (MHz):						11.0592					☒ Use On-chip ROM (0x0-0xFFFF)
Memory Model:		Small: variables in DATA									▼
Code Rom Size:		Large: 64K program									▼
Operating system:		None									▼

Step 5: Project  Click on Build target.

Build Output

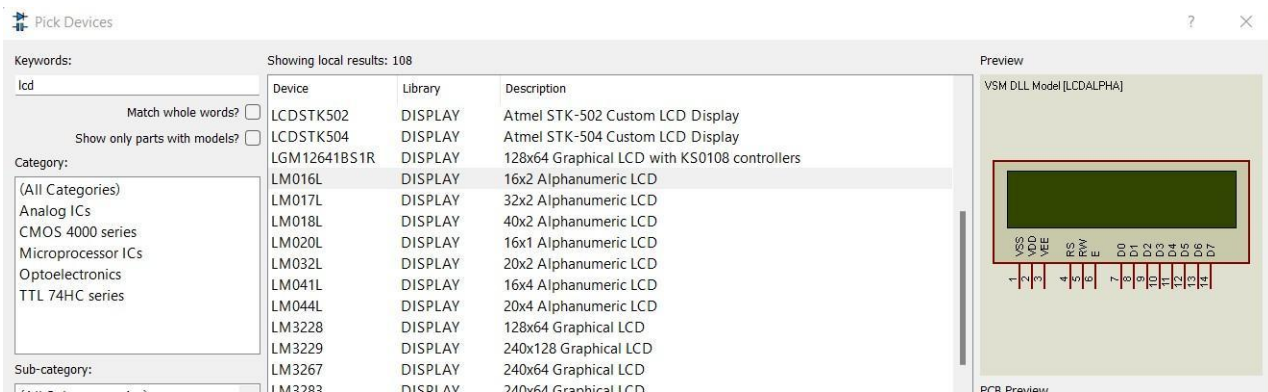
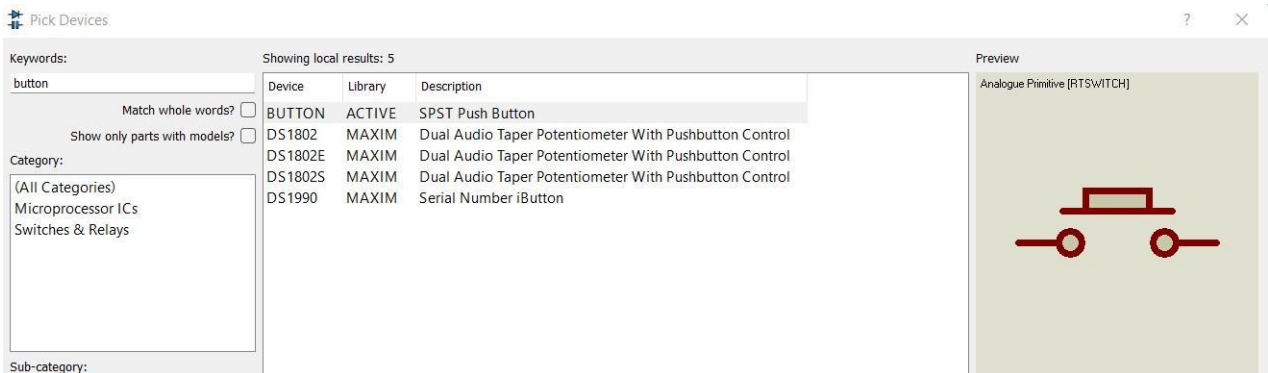
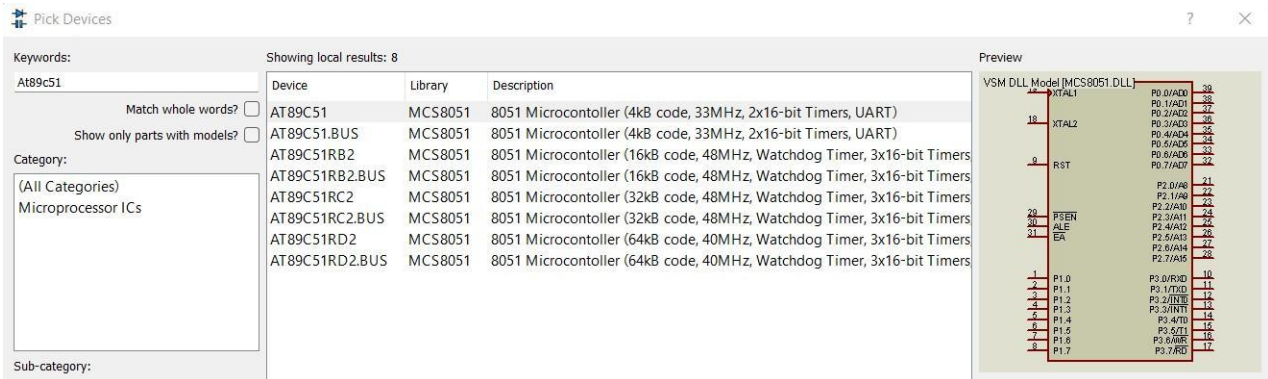
```
Rebuild started: Project: visitor counter
Rebuild target 'Target 1'
compiling visitor_counter.c...
linking...
Program Size: data=16.0 xdata=0 code=387
creating hex file from ".\Objects\visitor counter"...
".\Objects\visitor counter" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:00
```

Part 2: Designing circuit for Sensor based counting device in proteus.

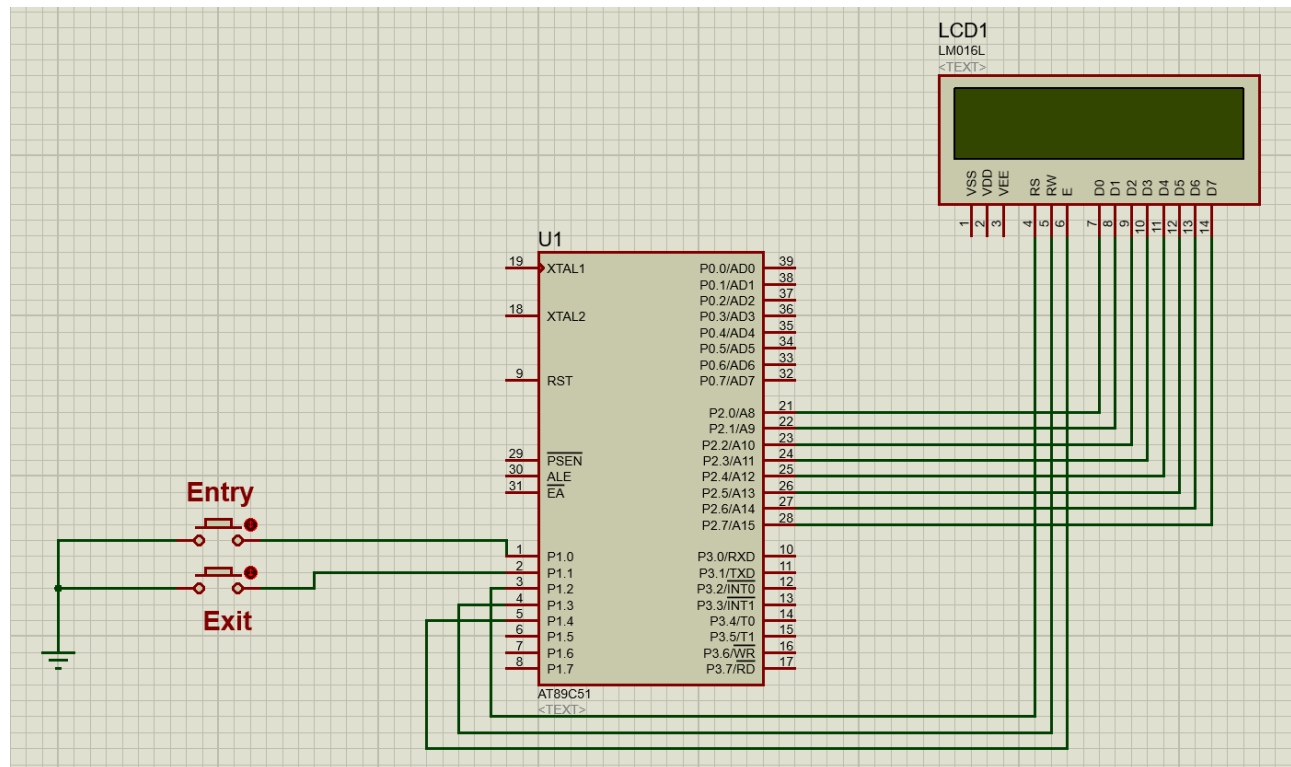
Step 1: Proteus 8 professional ? New project ? name for project ? Finish.

Step 2: From P (Device selector) search and add the components

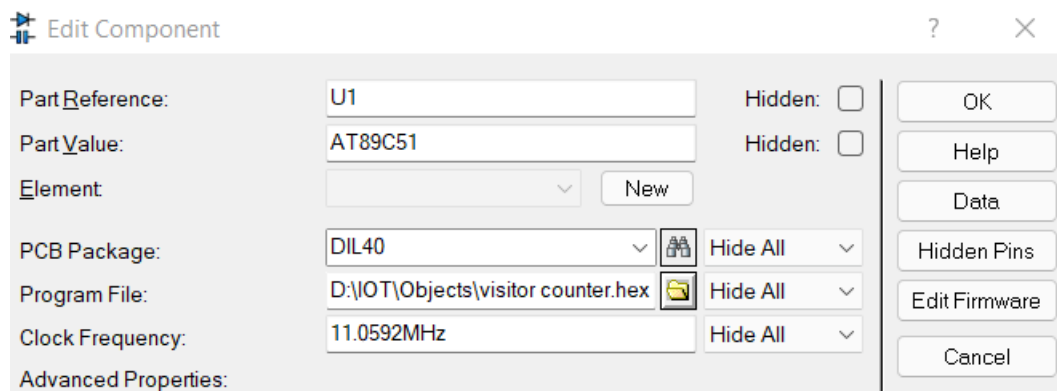
- AT89C51
- LCD: LM016L 16x2 Alphanumeric LCD
- Button (SPST Push button)



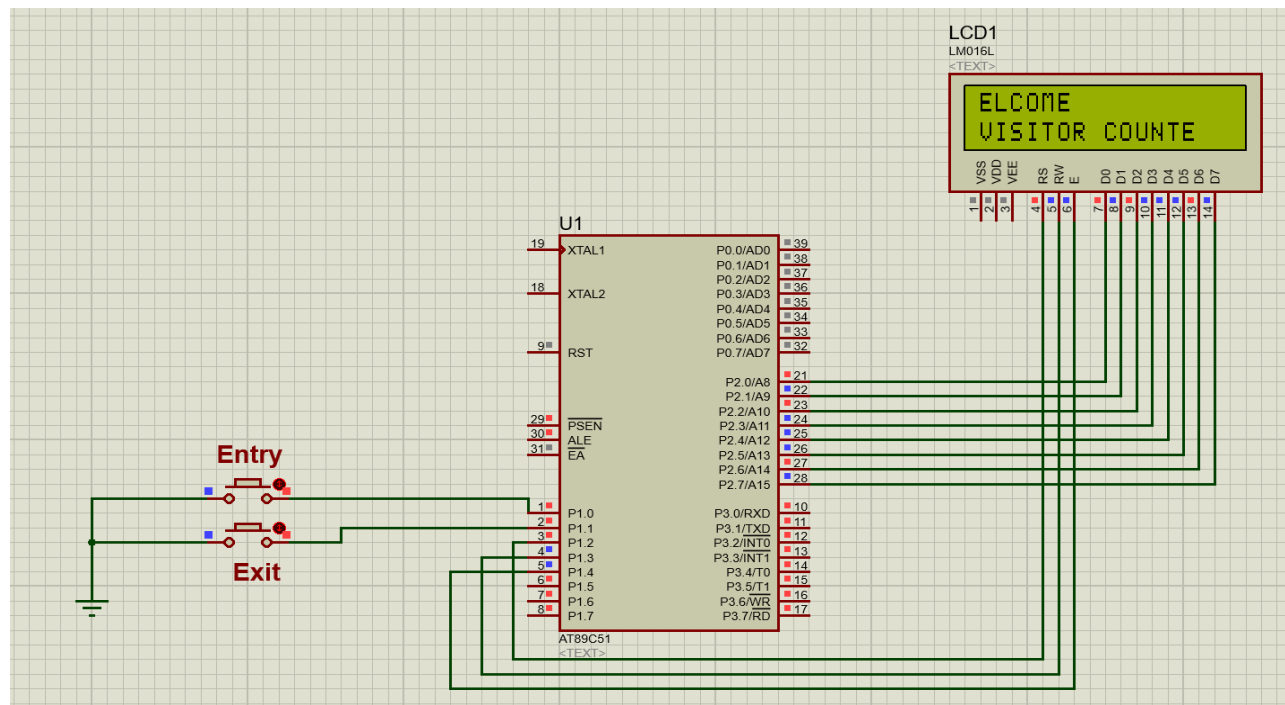
Step 3: Design the Circuit.



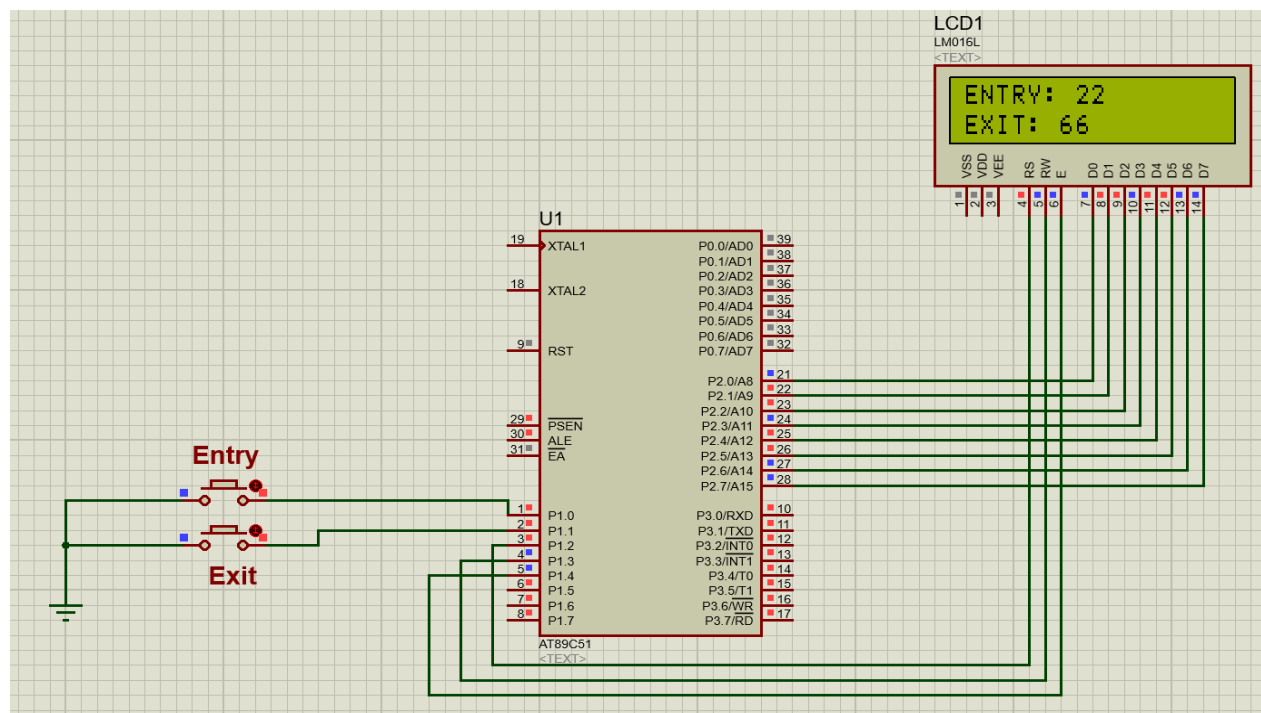
Step 4: Double click on Micro controller ? Select HEX file form program file ? Change Clock frequency to 11.0592MHz ? Click Ok



Output: Start the simulation.



- When entry button is pressed i.e., when visitor enters we get count of that on screen and in the same way for exit.

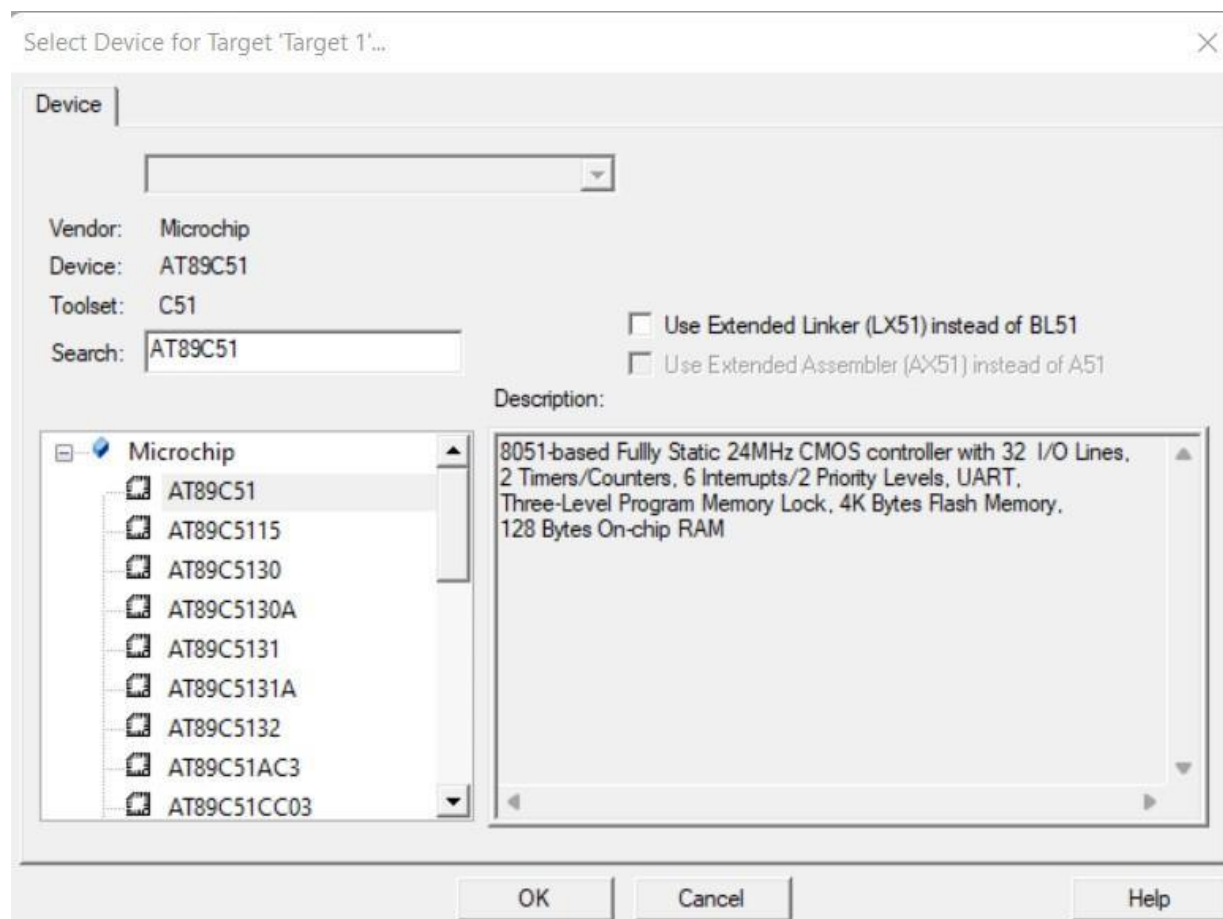


Practical: 2

Demonstrate communication between two embedded devices using UART port

Part 1: Writing C Program for Serial Communication using UART in Keil Software and creating Hex file.

Step 1: Project → New Project → Search your Folder → Give name to project → Select Atmel → AT89C51 → Click Ok.



Step 2: File ? New file ? Text Window is open ? Type program ? Save file by .c extension

Code:

```
#include<reg51.h>
```

```
void main()
```

```
{
```

```
/* Mode 1,
```

```
reception
```

```
enable */
```

```
SCON=0X50;
```

```
/* Timer 1, 8-
```

```
bit auto
```

```
reload mode
```

```
*/
```

```
TMOD=0X20;
```

```
TH1=-3; /*
```

```
Load value
```

```
baud rate */
```

```
TR1=1; /*
```

```
Start timer 1
```

```
*/
```

```
SBUF='I'; /*
```

```
Load char in
```

```

buffer */

while(TI==0);

TI=0;

}

```

Code 2 :

```

#include <reg51.h>          /* Include x51 header file */

void UART_Init()
{
    TMOD = 0x20;           /* Timer 1, 8-bit auto reload mode */
    TH1 = 0xFD;            /* Load value for 9600 baud rate */
    SCON = 0x50;           /* Mode 1, reception enable */
    TR1 = 1;               /* Start timer 1 */
}

void Transmit_data(char tx_data)
{
    SBUF = tx_data;        /* Load char in SBUF register */
    while (TI==0);        /* Wait until stop bit transmit */
    TI = 0;               /* Clear TI flag */
}

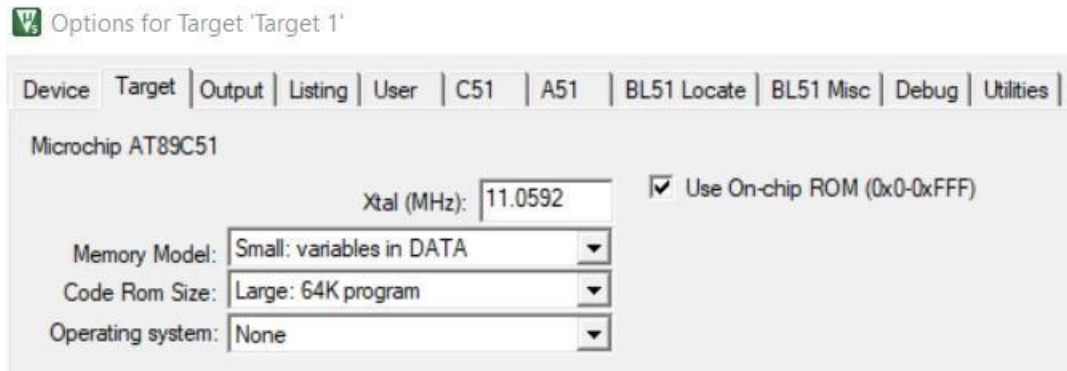
void String(char *str)
{
    int i;
    for(i=0;str[i]!=0;i++) /* Send each char of string till the NULL */
    {
        Transmit_data(str[i]); /* Call transmit data function */
    }
}

void main()
{
    UART_Init();           /* UART initialize function */
    String("test");        /* Transmit 'test' */
    while(1);
}

```

Step 3: Click on target ? Source group ? right click ? Add existing file ? select program file ? Click on Add.

Step 4: Right click on target ? Option for target ? Change frequency to 11.0592 MHz ? Click on use on-chip ROM ? Go to output label ? Click on Create Hex file ? Ok.



Step 5: Project ? Click on Build target.

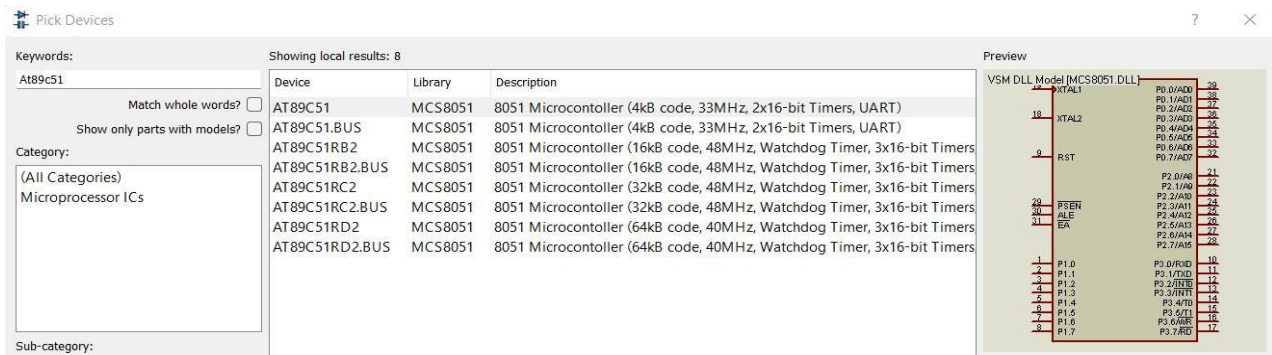
```
Build Output
Rebuild started: Project: uart
Rebuild target 'Target 1'
compiling uart.c...
linking...
Program Size: data=9.0 xdata=0 code=35
creating hex file from ".\Objects\uart"...
".\Objects\uart" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:00
```

Part 2: Designing circuit for Serial communication using UART in proteus.

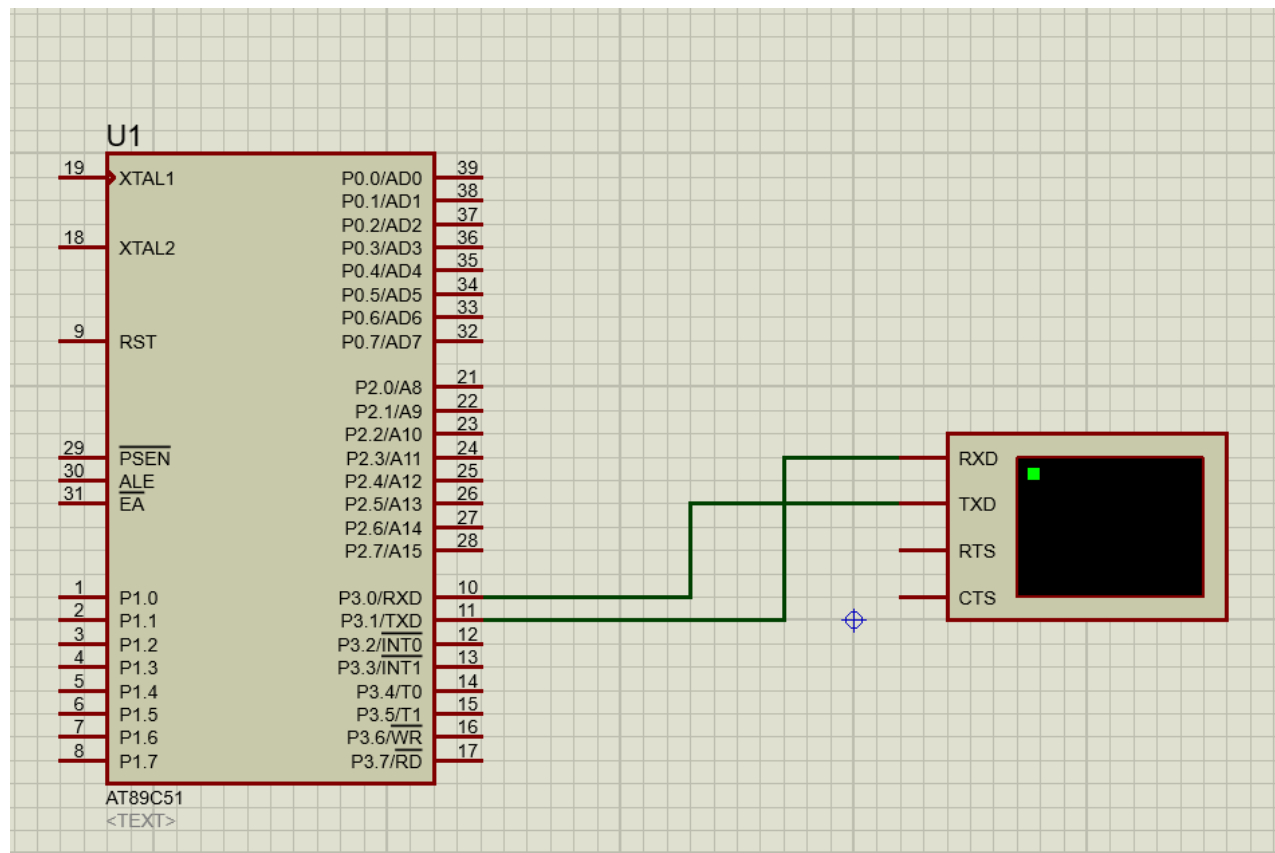
Step 1: Proteus 8 professional ? New project ? name for project ? Finish.

Step 2: From P (Device selector) search and add the components

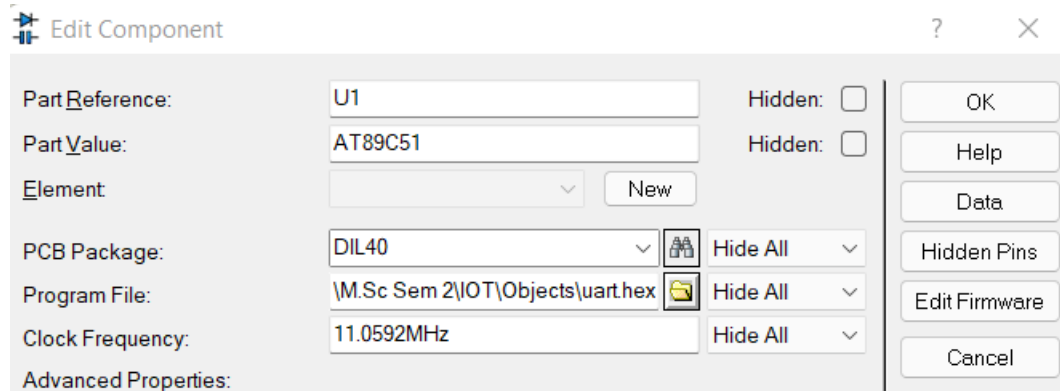
- AT89C51
- Virtual Terminal from instruments.



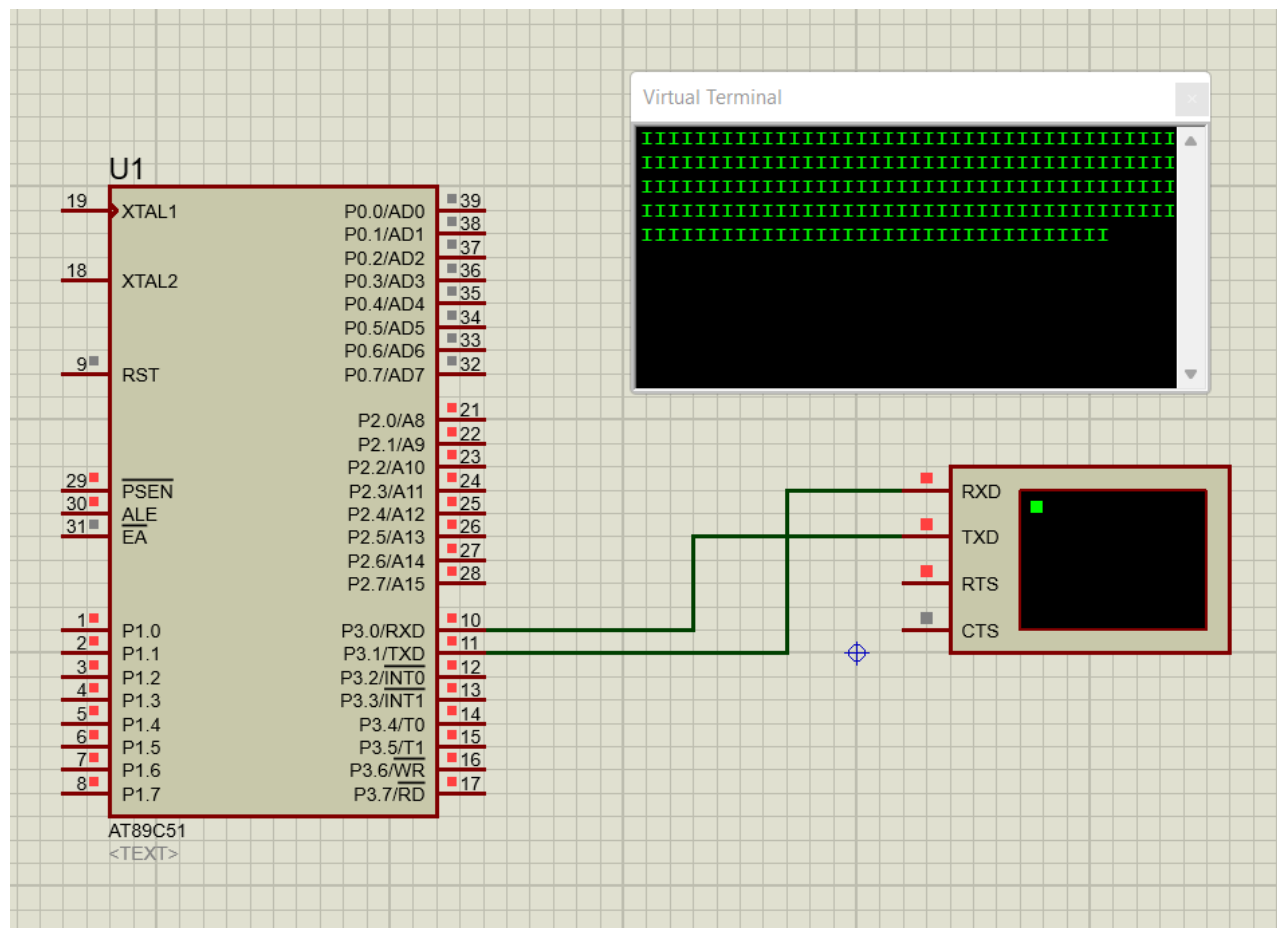
Step 3: Design the Circuit.



Step 4: Double click on Micro controller ? Select HEX file form program file ? Change Clock frequency to 11.0592MHz ? Click Ok



Output: Start the simulation.



Practical: 3

Built an IoT system to send ticket before entering the bus

Part 1: Writing C Program in Arduino IDE and creating Hex file.

Step 1: Open Arduino → New file → Write program → Save file.



The screenshot shows the Arduino IDE interface. At the top, the title bar reads "rfid | Arduino 1.8.16". Below it is a menu bar with "File", "Edit", "Sketch", "Tools", and "Help". A toolbar with icons for opening, saving, and running is visible. The main text area contains the following C code:

```
#include <Servo.h>

int count = 0;
char c;
String id;
Servo myservo;

void setup() {
  Serial.begin(9600);
  myservo.attach(9);
  myservo.write(0);
  pinMode(13, OUTPUT);
  Serial.println("Please scan your RFID TAG");
}

void loop() {
  while(Serial.available()>0)
  {
    c = Serial.read();
    count++;
    id += c;
  }
}
```

Code:

```
#include <Servo.h>

int count = 0;
char c; String
id;
Servo myservo;

void setup() {
  Serial.begin(9600);
  myservo.attach(9);
  myservo.write(0);
  pinMode(13, OUTPUT);
  Serial.println("Please scan your RFID TAG");
}

void loop() {
  while(Serial.available()>0)
  {
    c = Serial.read();
    count++;
    id += c;
    if(count == 12)
    {
      Serial.print(id);
      //break;

      if(id=="AB123456789A") //Define valid tag id here
      {
        Serial.println("Valid TAG");
        digitalWrite(13, HIGH);
        myservo.write(180);
        delay(4000);
        myservo.write(0);
      }
    }
  }
}
```



```

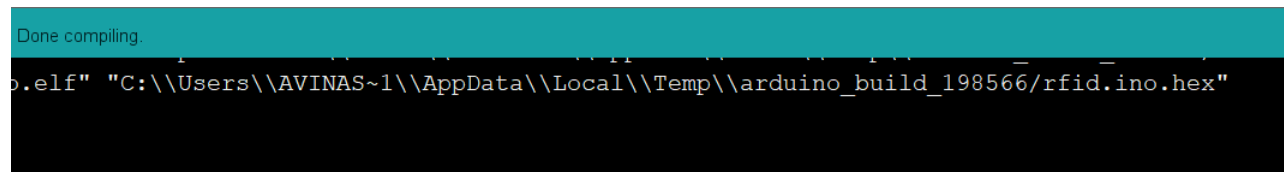
    }
    else
    {
        digitalWrite(13, LOW);
        Serial.println("Invalid TAG");
        myservo.write(0);
    }
}
}
count = 0;
id="";
delay(500);

}

```

Step 2: Click on Verify option and verify/compile the code.

Step 3: After verifying the program select path of the hex file and copy that path and paste it the Proteus file.



Part 2: Designing circuit in proteus.

Note : If Arduino is not added in proteus, download Arduino library. Right click C drive, and choose show hidden folder. Now open program data and copy the .lib files in the library folder .

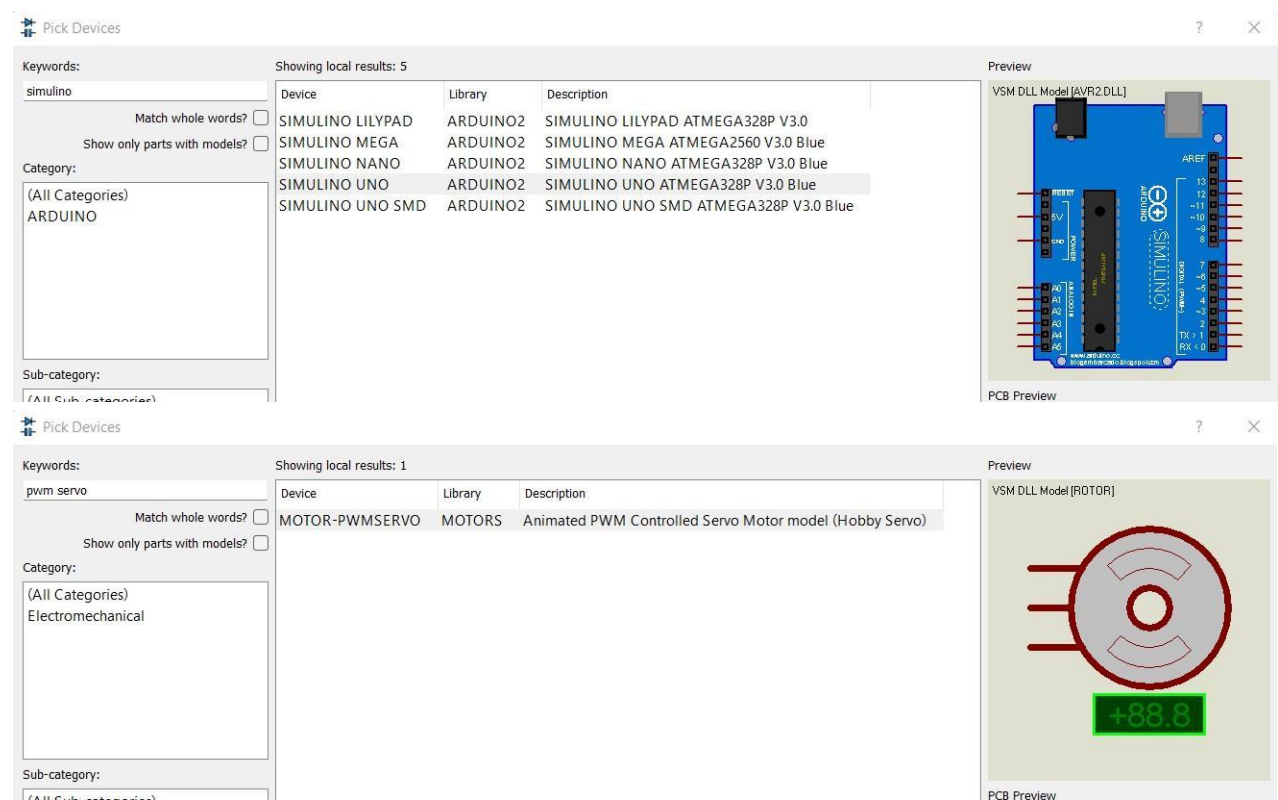
AB123456789A

1234567

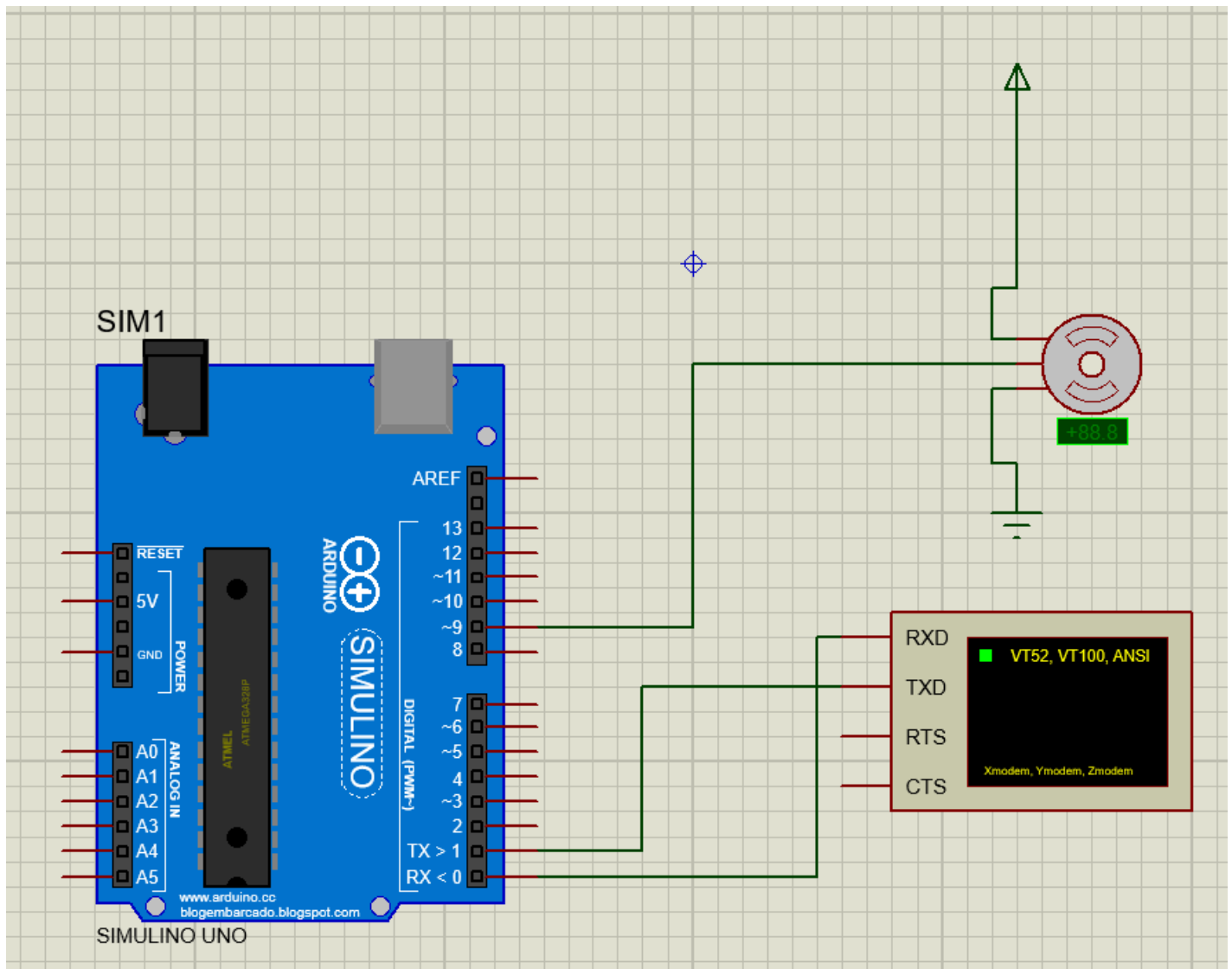
Step 1: Proteus 8 professional ? New project ? name for project ? Finish.

Step 2: From P (Device selector) search and add the components

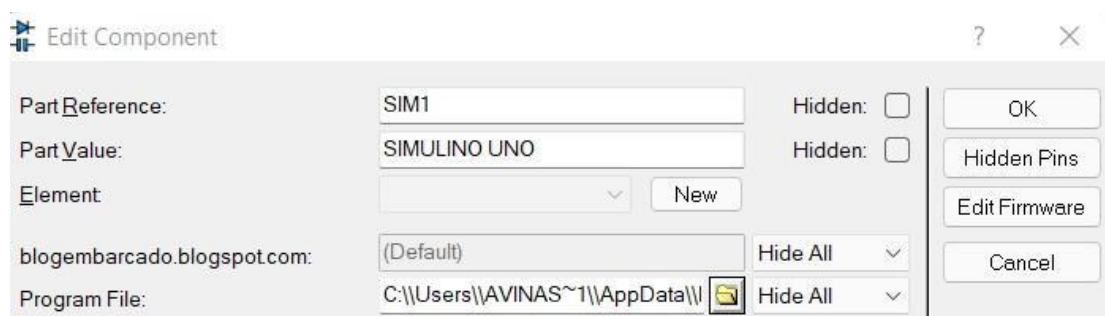
- ☐ SIMULINO UNO controller
- ☐ MOTOR- PWMSERVO -Animated PMW Controlled Servo motor
- ☐ Virtual Terminal from instruments.



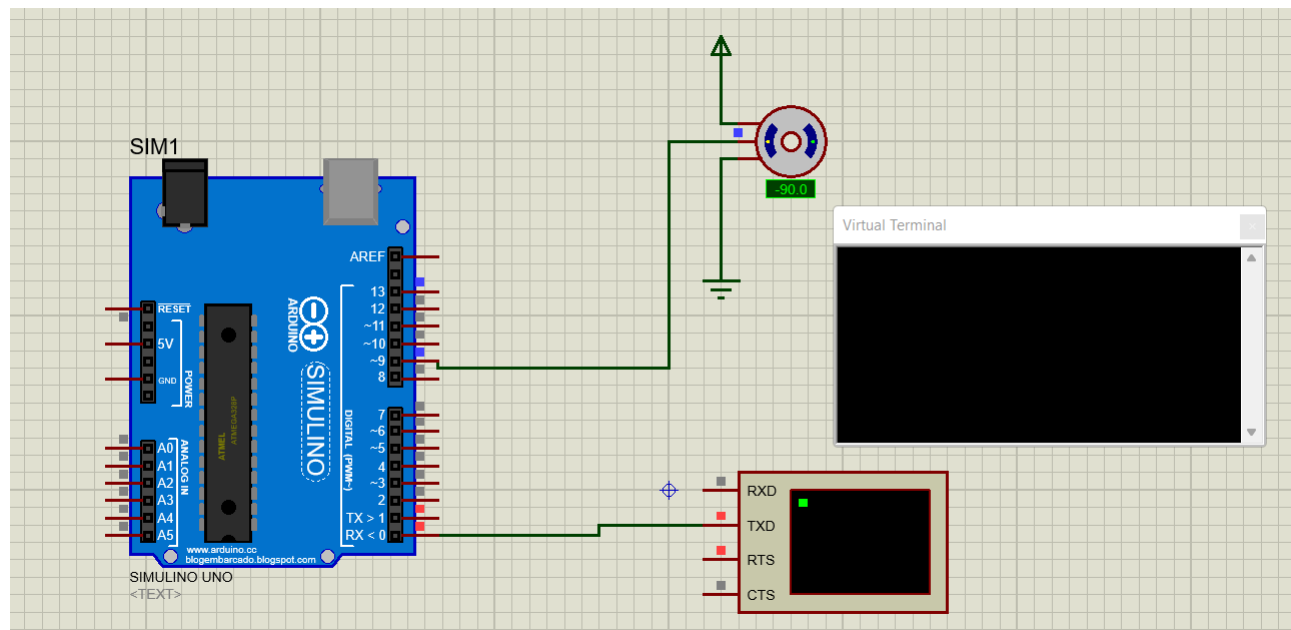
Step 3: Design the Circuit.



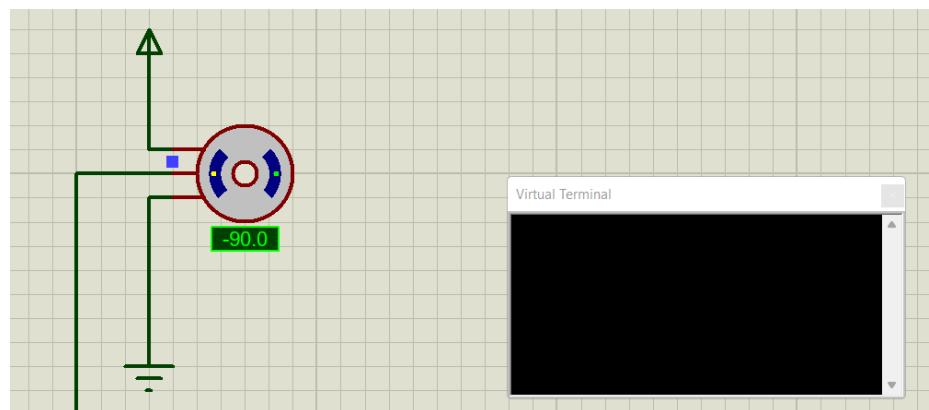
Step 4: Double click on controller Select HEX file form program file Click Ok



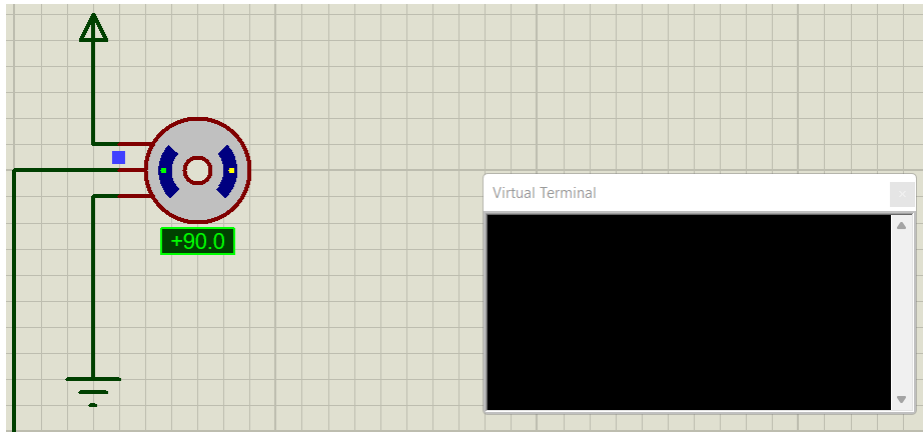
Output: Start the simulation.



- When we start the simulation the servo motor is at -90 degree i.e., the door is closed.



- When we put the ticket code in virtual terminal it checks that it is valid or not, if the code is valid the door should open i.e., we can see the servo motor angle changed to +90 degree after some time it goes back to it's initial position.



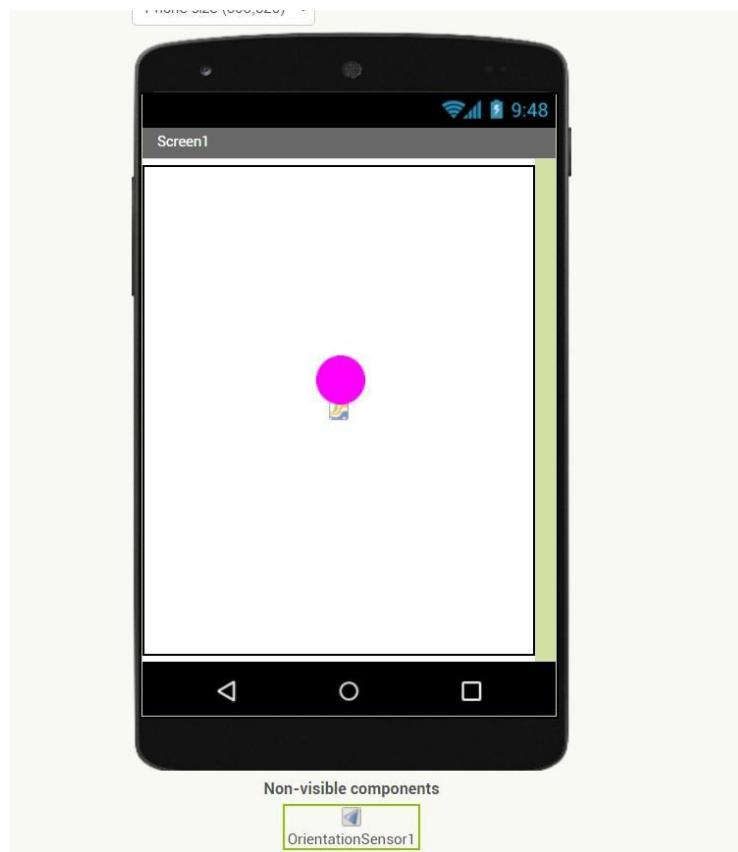
Practical: 4

Develop a IoT application which will record the movement and orientation of your phone and give the data back to the PC

Step 1: Go to your Browser → Search MIT App Inventor → Create apps → new project → click on continue → my project → start new project → give project name → click ok.

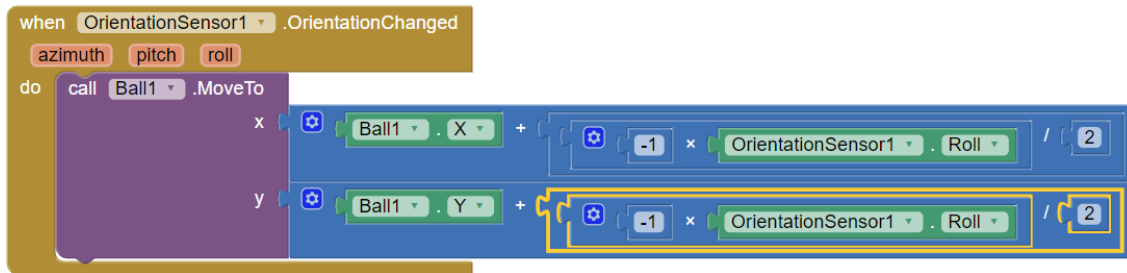
Step 2: Designer part

From palette → go to sensor → select orientation sensor → go to drawing and animation → select canvas → take a ball and put it in canvas.



Step 3: Block part

Click on orientation sensor  and select 1st option i.e. When orientation change. ball  select call. move to x to y  Then in x put math function i.e., $\text{Ball } x + (-1) * \text{orientation. sensor roll} / 2$  Then similarly put for y i.e., $\text{Ball } y + (-1) * \text{orientation sensor pitch} / 2$.



Step 4: Run part

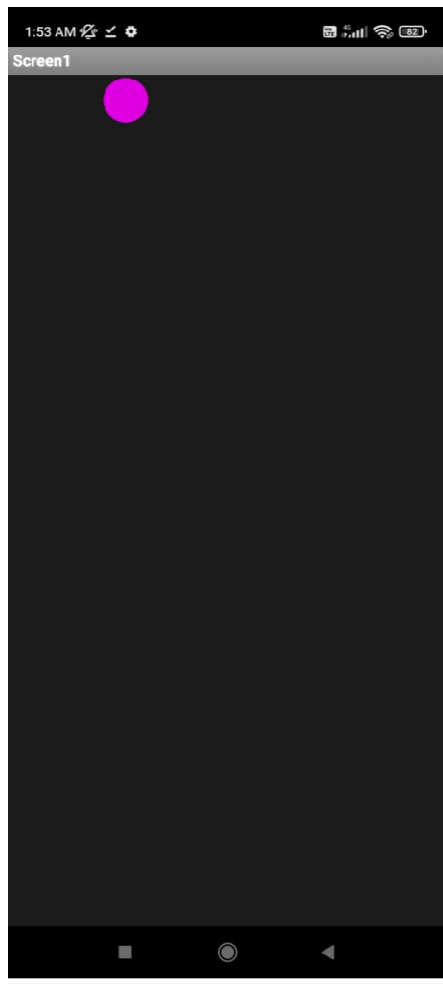
Project  save project

Download MIT app i.e. (MIT A12 Companion) on your phone.

Build -> Android app ".apk" then scan the code and get output.

OUTPUT:

 Here, we can see the orientation of the phone, with the help of ball.



Practical: 5

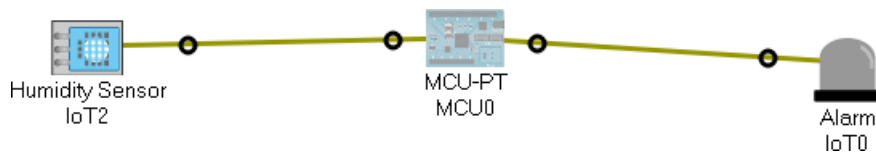
Develop an IoT application that will raise an alarm whenever with going to rain outside

Step 1: Open Cisco Packet Tracer.

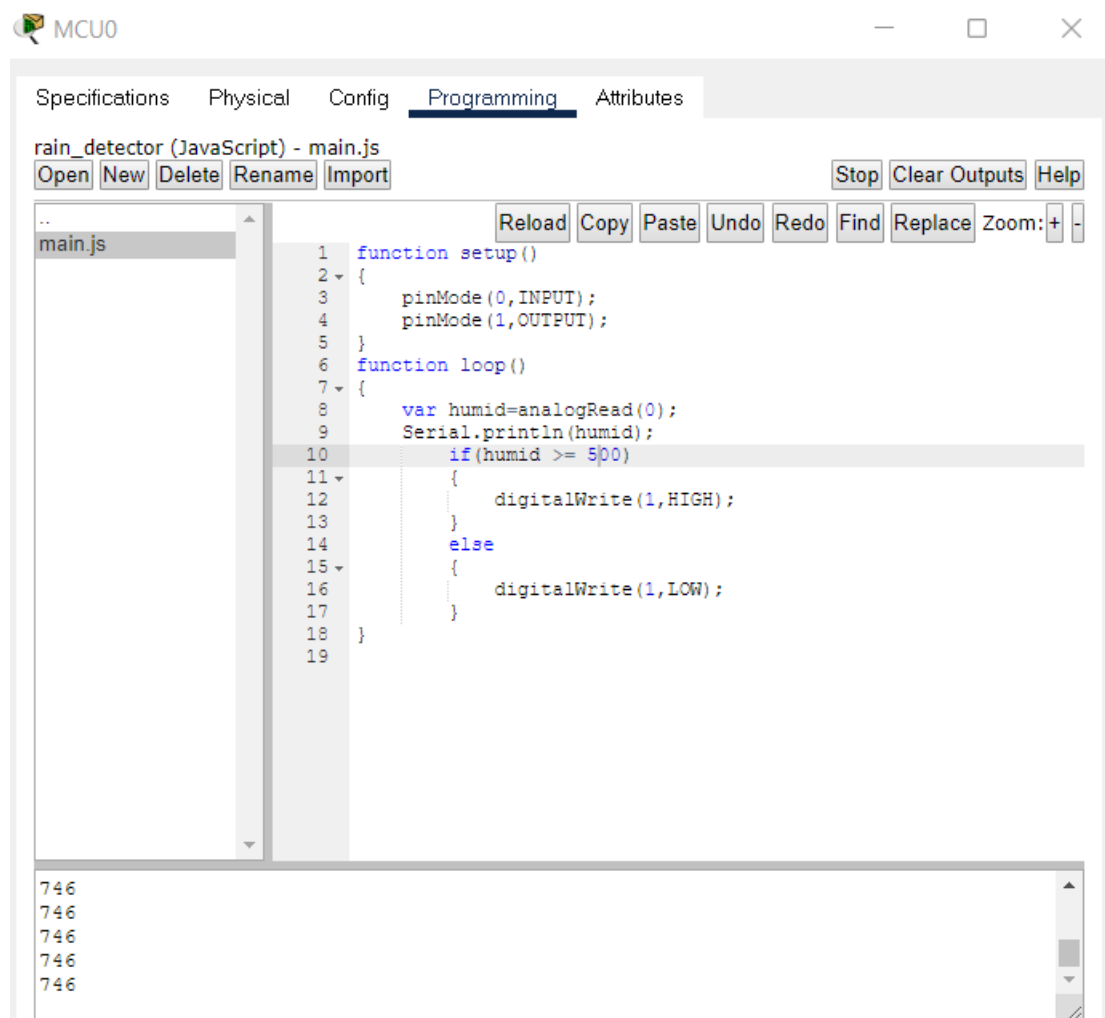
Step 2: Design the circuit.

Step 3: Search and take components Humidity sensor, MCU-PT and Alarm.

A0 pin of MCU connect with **A0 pin of Humidity sensor** and **D1 pin of MCU** connect with **D0 pin of Alarm** using IOT custom cable.



Step 4: Select MCU-PT and write click on it ? Programming ? New button ? Create a new file (motion detection.js) ? Write a motion detection program.



Code:

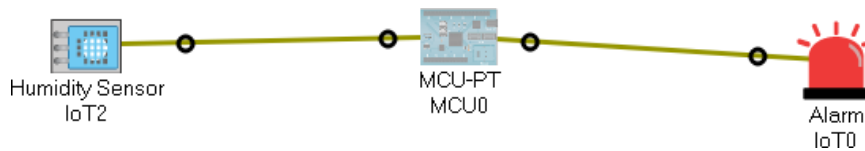
```
function setup()
{
  pinMode(0, INPUT);
  pinMode(1, OUTPUT);
}
function loop()
{
  var humid = analogRead(0);
  Serial.println(humid);
  if(humid >= 500)
  {
    digitalWrite(1, HIGH);
  }
}
```

```
    }  
    else  
    {  
        digitalWrite(1,LOW);  
    }  
}
```

Step 5: Click on run button and run the program.

OUTPUT:

- Here, we can see when humidity level is greater than 500 i.e., when it is raining outside the Alarm is turned ON.



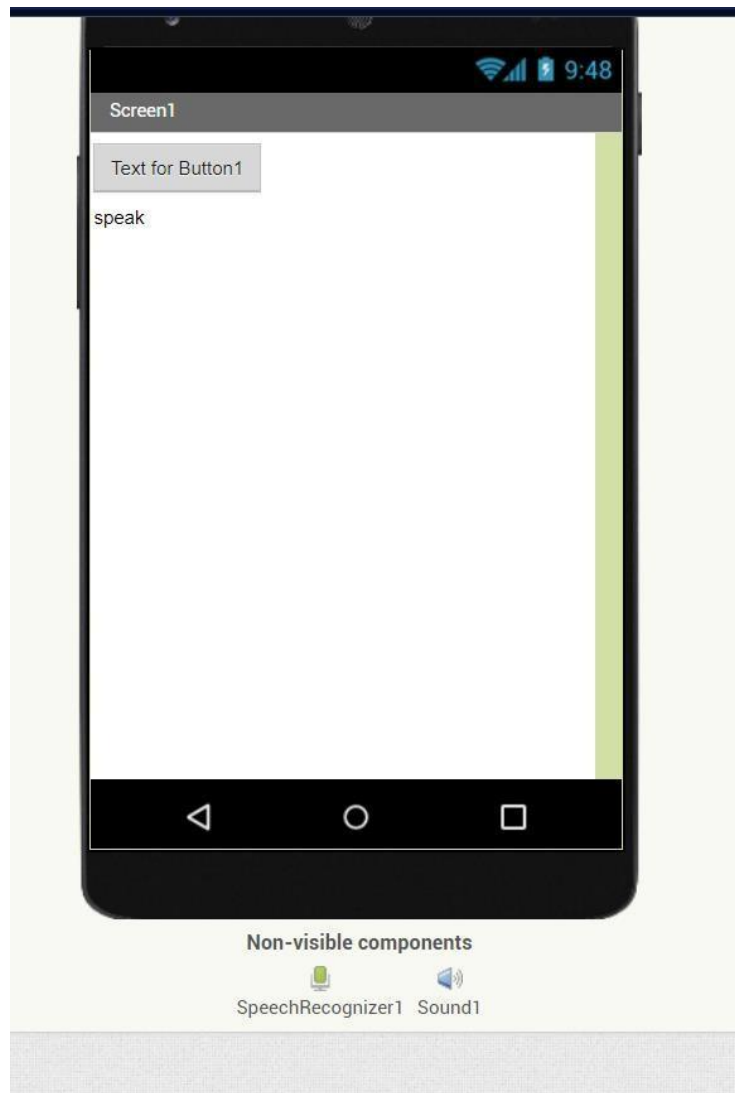
Practical :6

Deploy an IoT application which will alert you by beeping or vibrating your phone whenever you get someone call your name

Step 1: Go to your Browser → Search MIT App Inventor → Create apps → new project → click on continue → my project → start new project → give project name → click ok.

Step 2: Designer part

From palette → go to user interface → select button (change properties i.e., button text as speak) → select label (change properties i.e., label text as click on speak button)
Go to media → select speech recognition → select sound.



Step 3: Block part

a) Click on button and select When do. Click option

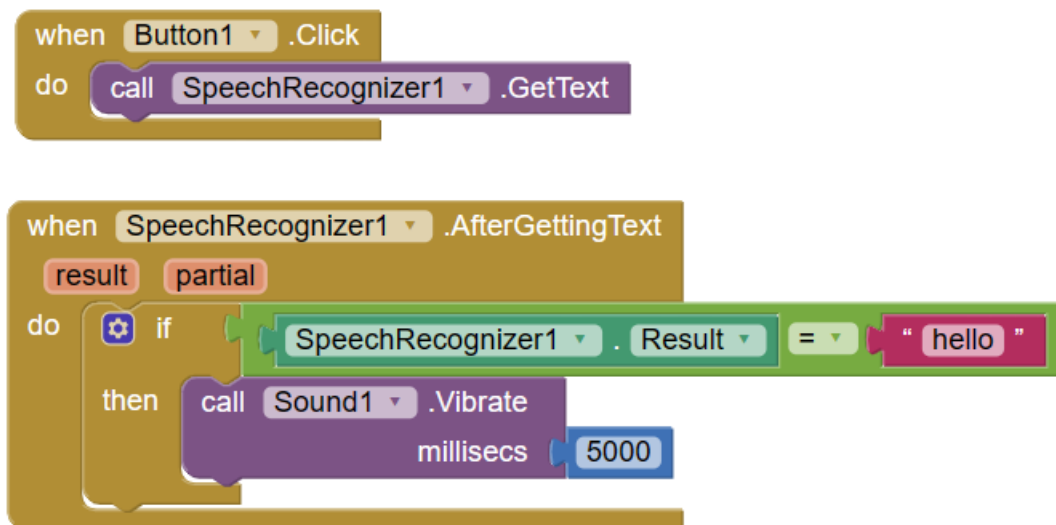
b) speech recognition select call. Get text

c) sound recognition select 1st option i.e. After getting text

d) control select if and then option

In if condition: 1st select math i.e., 3rd option with= condition, 2nd select speech i.e., speech. Result and put in if condition 1st part, 3rd select text i.e., select 1 option of text and put it in if condition 2nd part (as your name).

In then part select sound i.e., vibrate mileage in this add math 1st option and add value as 2000.



Step 4: Run part

Project save project

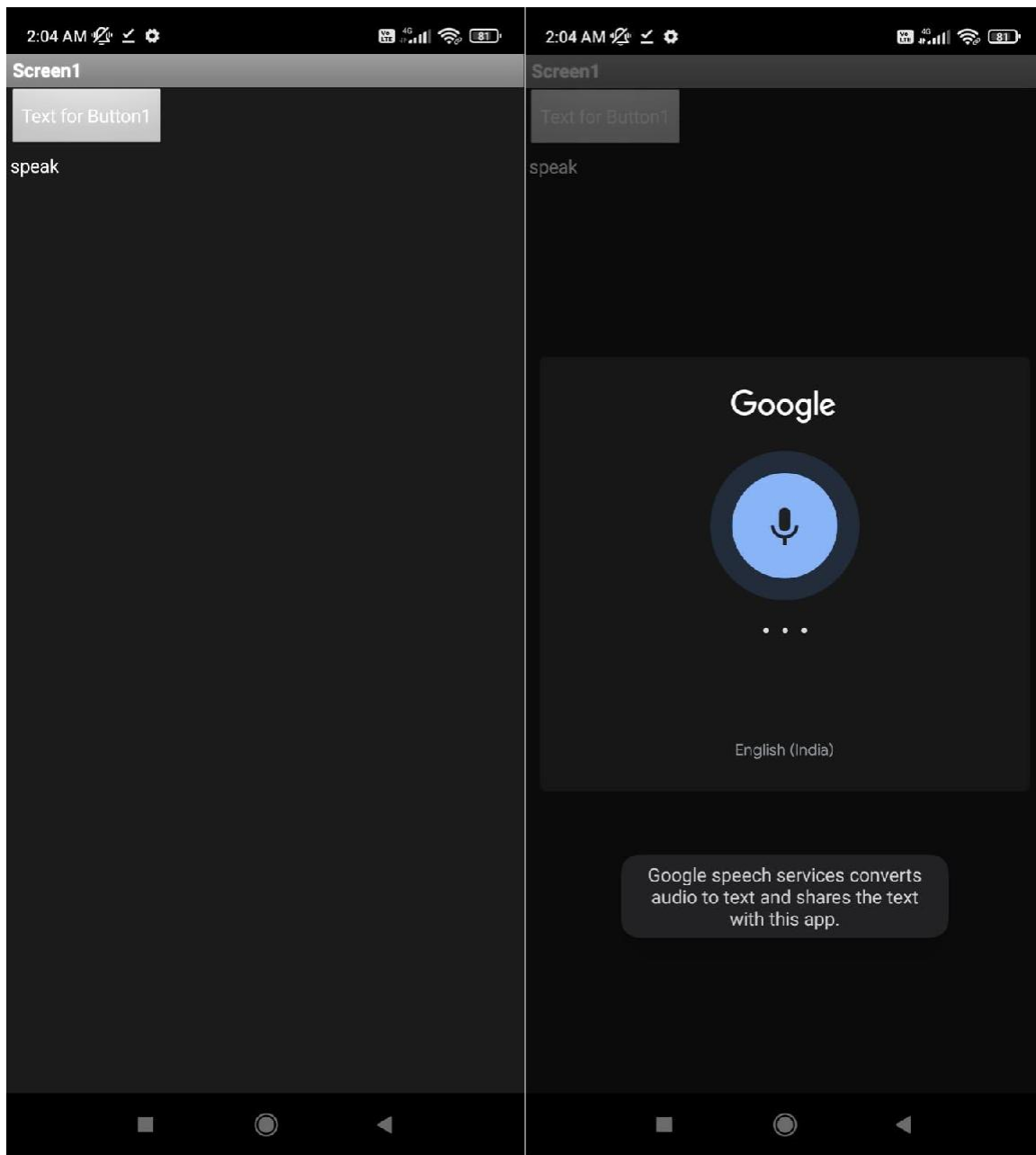
Download MIT app i.e. (MIT A12 Companion) on your phone.

Build Android app apk then scan the code and get output.

As you speak your name your phone gets vibrate.

OUTPUT:

- Here, we can see the when the button is pressed on the phone, mic for recording the voice comes and if it records the audio as “Avinash” the phone will vibrate.



2:04 AM

4G 47% 81

Screen1

Text for Button1

speak

Google



English (India)

Practical: 7

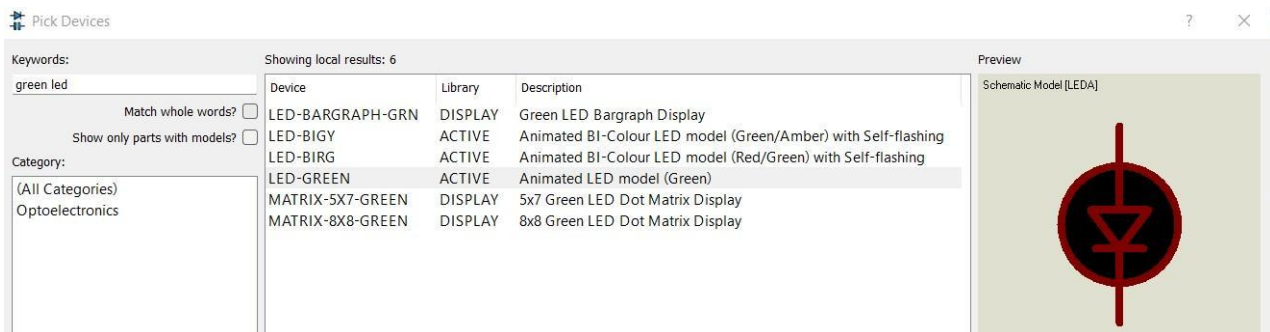
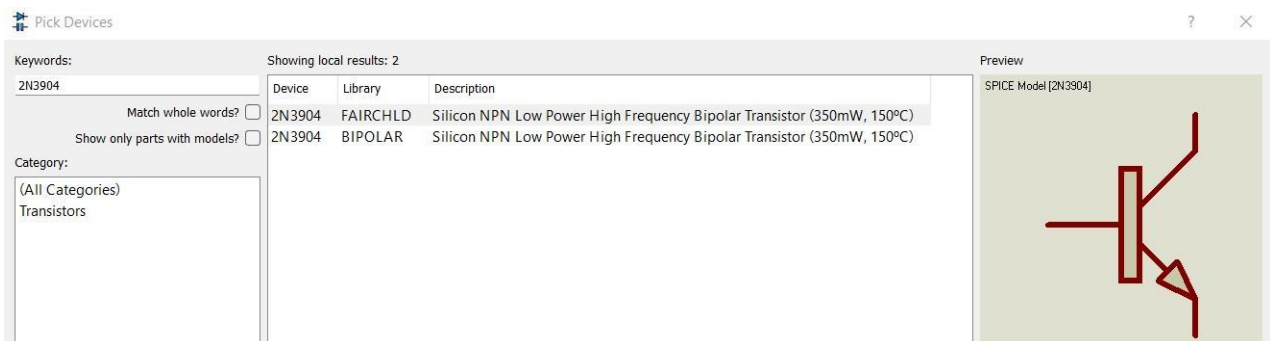
Develop an IoT application for monitoring water levels in tanks

□ Designing circuit in proteus.

Step 1: Proteus 8 professional → New project → name for project → Finish.

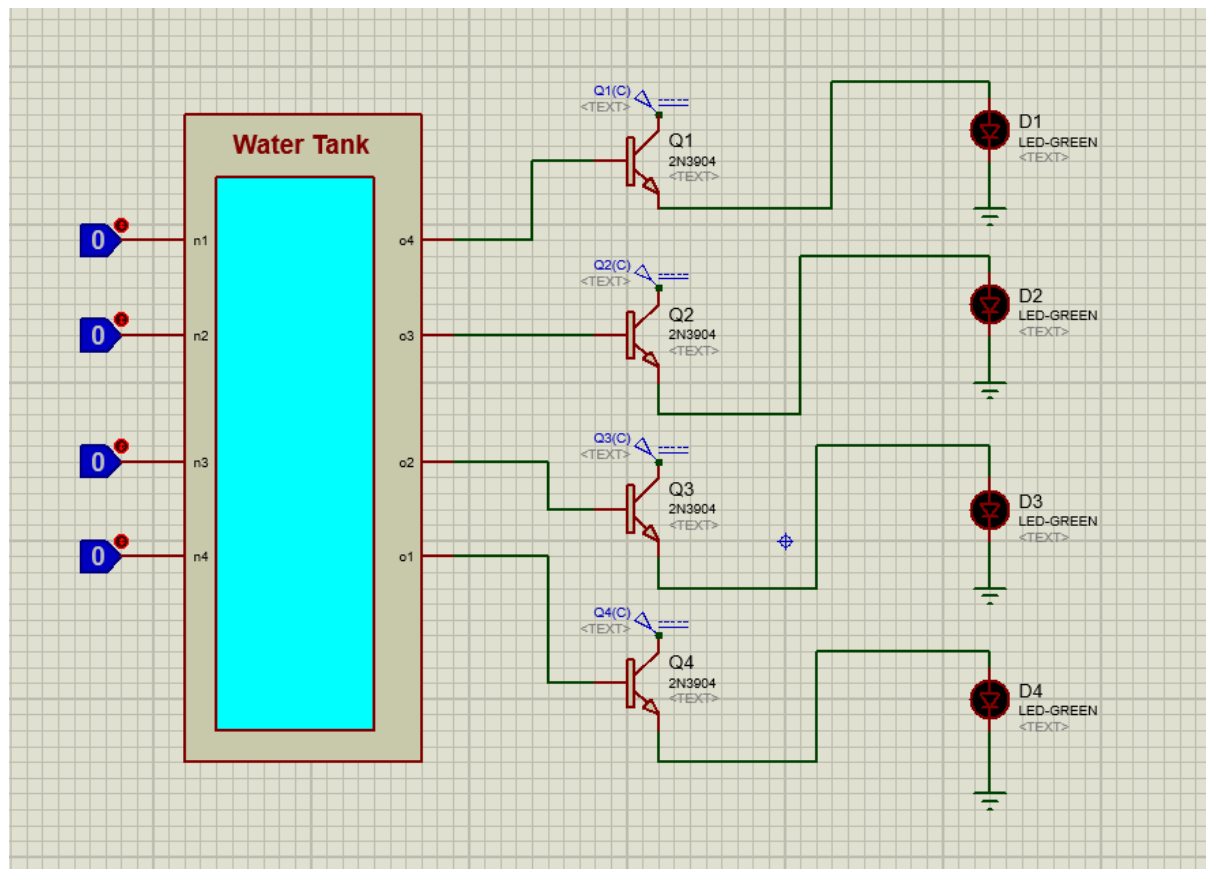
Step 2: From P (Device selector) search and add the components

- 2N3904 – 4 placed, Silicon NPN Low Power High Frequency Bipolar Transistor
- LED GREEN – 4 placed, Animated LED model (Green)
- LOGICTOGGLE – 4 placed, Logic State Source (Momentary Action)

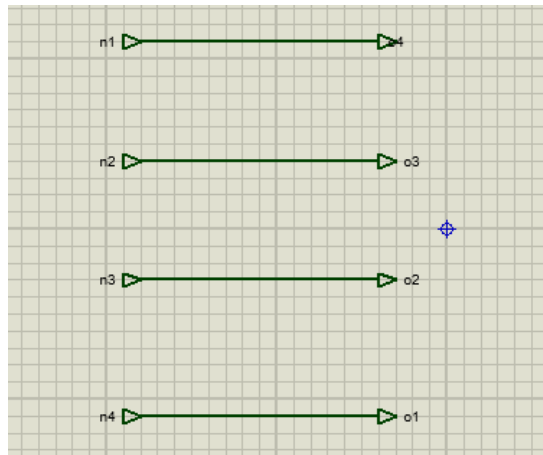


Step 3: Create a water tank device using components in graphics, right click on it ☐ make device, give name of device (water level tank) ☐ Next ☐ Device category – Mechanics ☐ Click on the water level tank and go to edit properties click on it attach hierarchy module, ok ☐ again right on it and select the go to child sheet ☐ In the child sheet take 4 input terminals and 4 output terminals and connect it.

Step 4: Design other circuit as shown below.

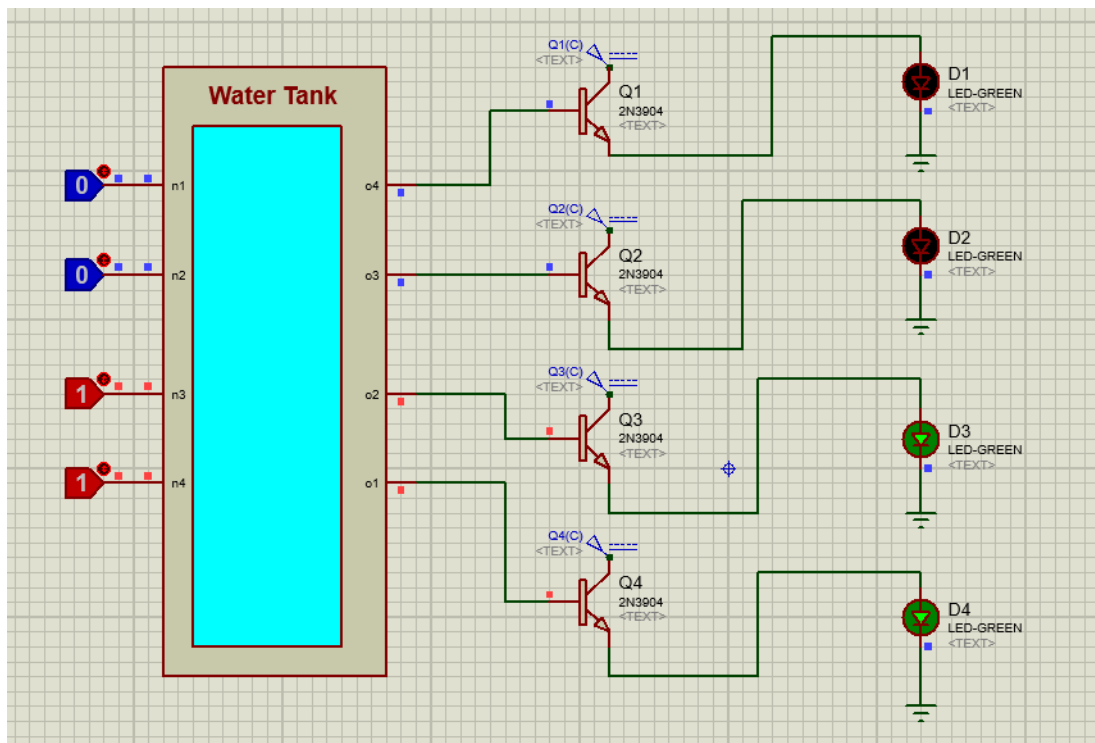


Step 5: Right Click on Water Tank go to child sheet design the logic.

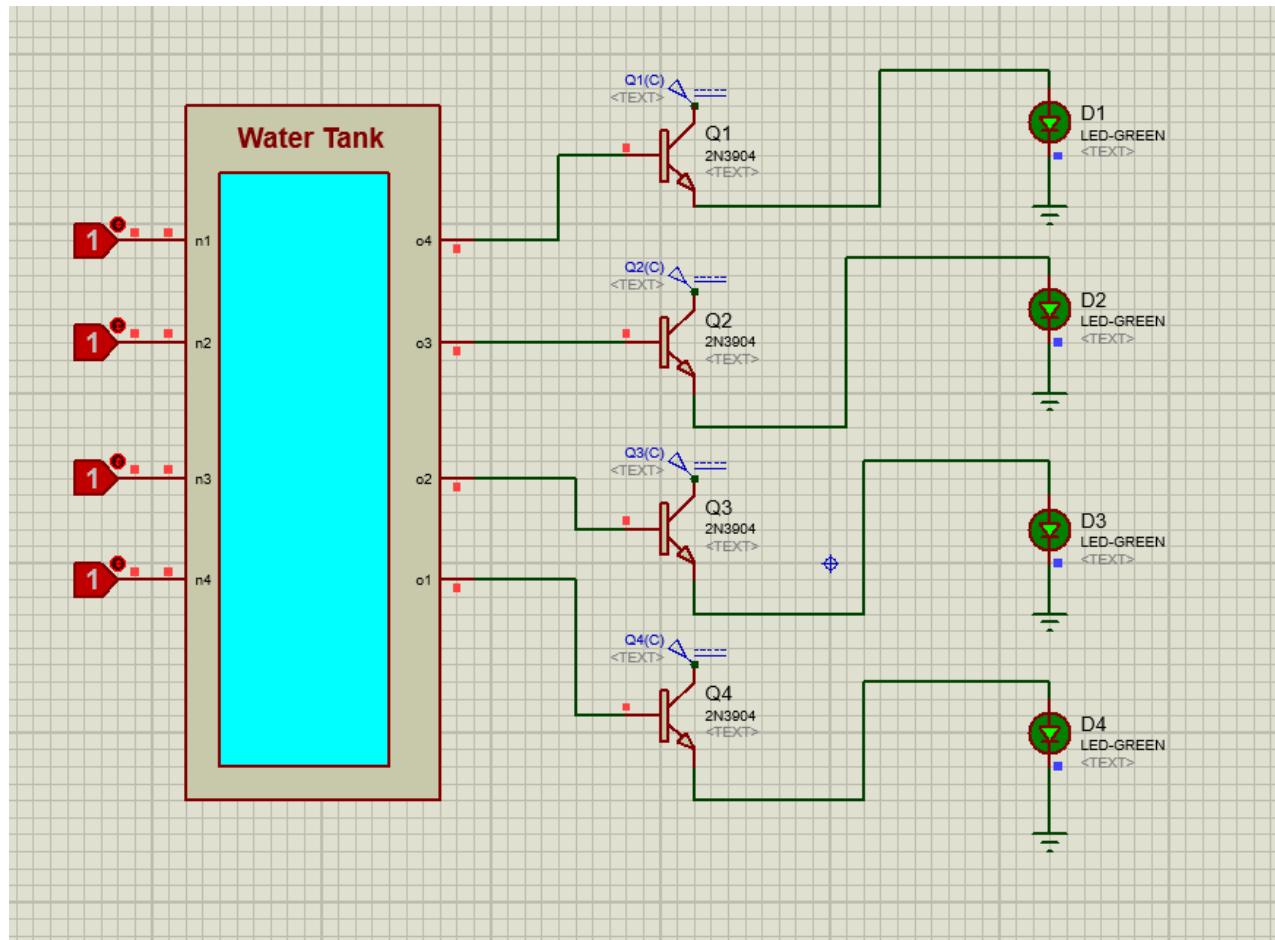


Output: Start the simulation.

□ When the water level is at 2, we can see the bottom 2 led are ON.



□ When water level is full all the led's are ON.



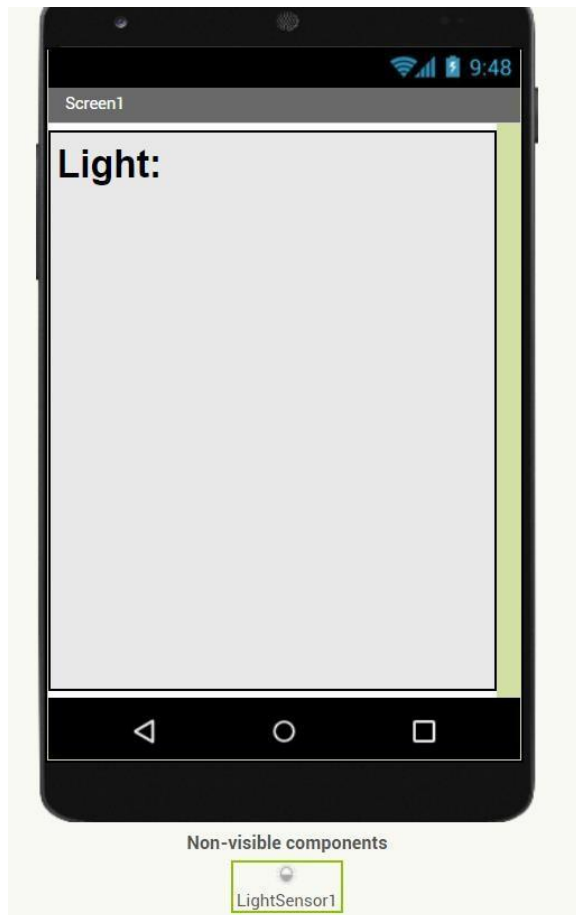
Practical: 8

Develop an IoT module to which measure the intensity of light and send the same to your PC/ Phone

Step 1: Go to your Browser → Search MIT App Inventor → Create apps → new project → click on continue → my project → start new project → give project name → click ok.

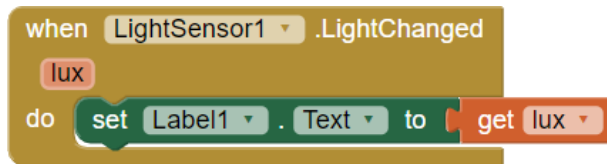
Step 2: Designer part

From palette → go to layout → select vertical arrangements (change properties: heights, width should be parent) → go to user interface → select label (change properties: font bold and font size, Text alignment should be center) → go to sensor → drag and drop light sensor.



Step 3: Block part

- Click on light sensor → drag and drop 1 option when. Light changed part.
- Label → drag and drop label1. Text to.
- then click on lux → select get lux → and connect to label1. Text to.

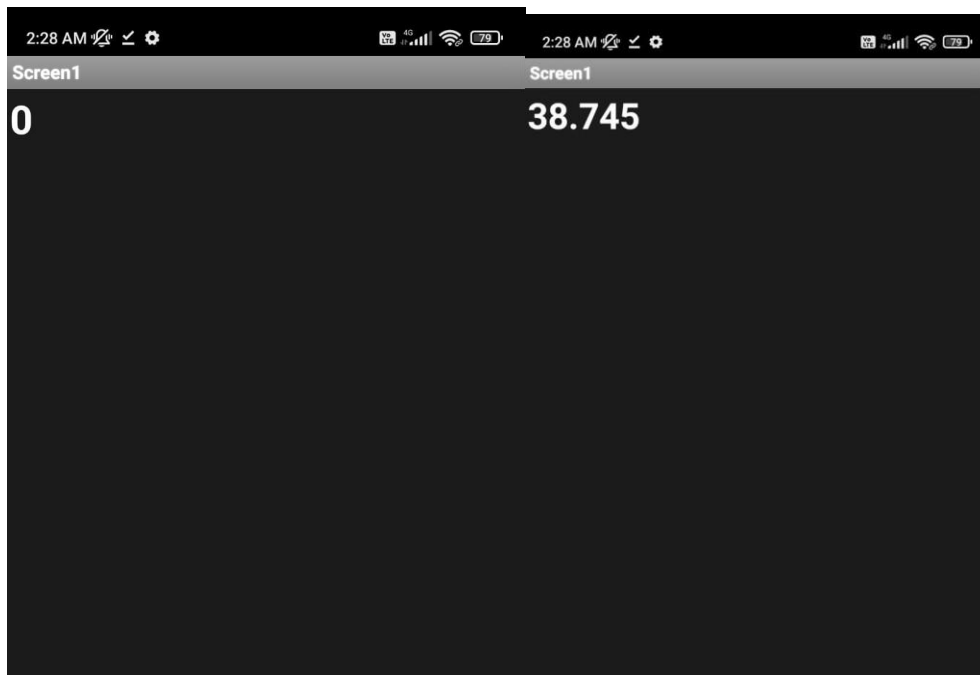


Step 4: Run part

Project → save project.
Download MIT app i.e. (MIT A12 Companion) on your phone.
Build -> Android app apk then scan the code and get output.

OUTPUT:

- Here, when we go to dark place we get intensity as 0 and when we go in light we get some values i.e., the intensity of light at that place.



Practical: 9

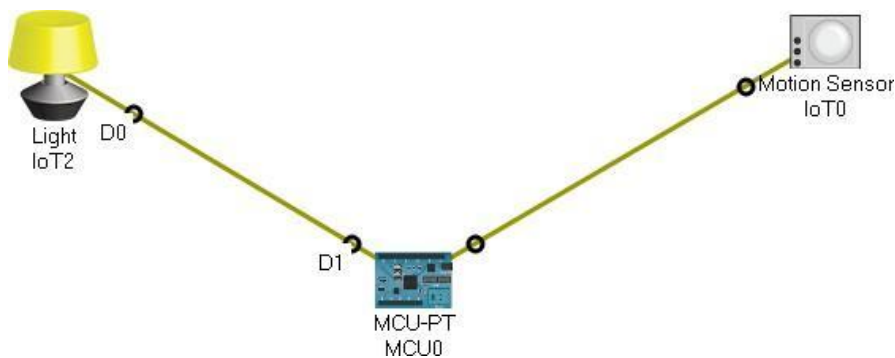
Develop an IoT application for Motion detection

Step 1: Open Cisco Packet Tracer.

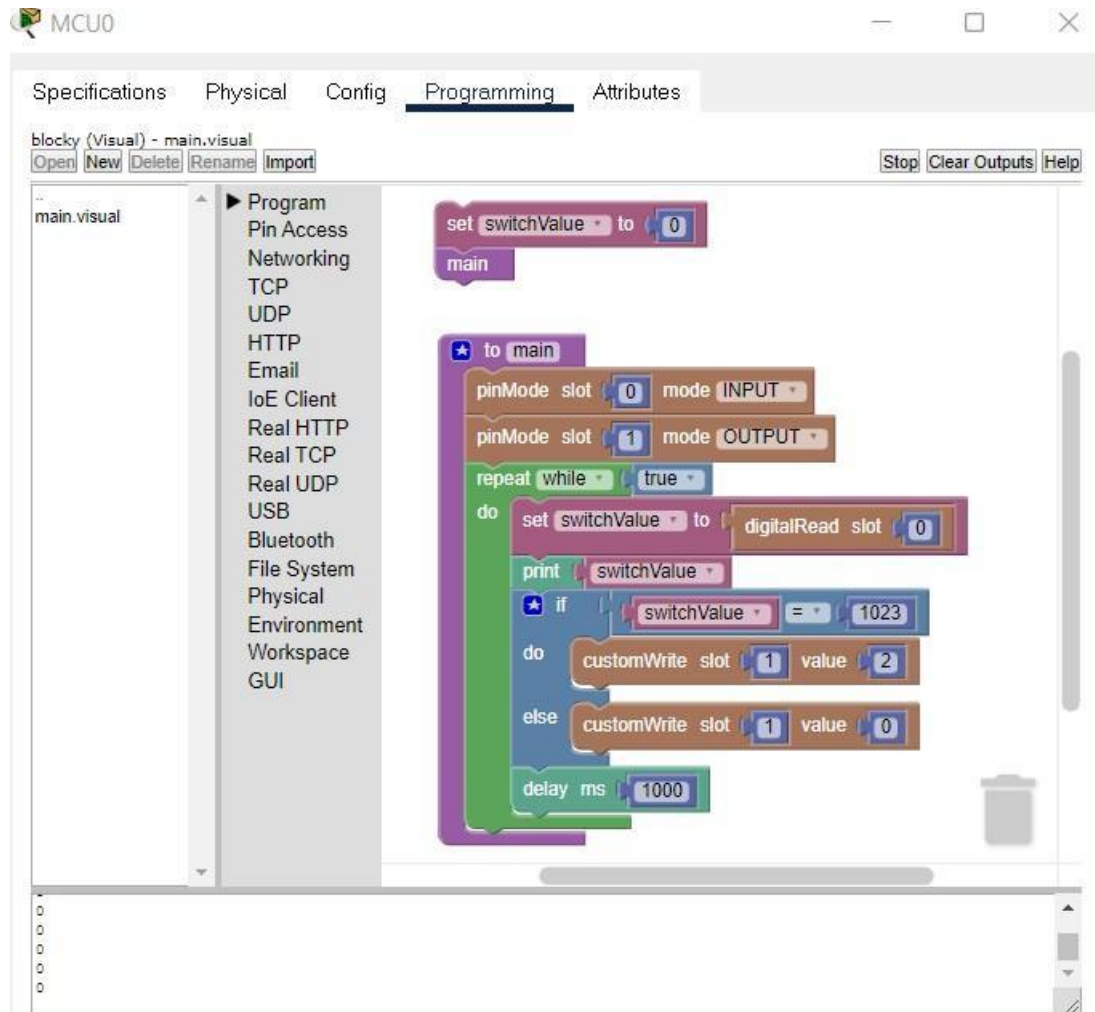
Step 2: Design the circuit.

Step 3: Search and take components Motion sensor, MCU-PT and Light.

D0 pin of MCU connect with **D0 pin of Motion** sensor and **D1 pin of MCU** connect with **D0 pin of light** using **IOT custom cable**.



Step 4: Select MCU-PT and write click on it ? Programming ? Select blocky(visual) ? Create a new file and use blocks to write the code.



Step 5: Click on run button and run the program.

Step 6: Click Alt and move mouse on the motion sensor and view the output of Light.

OUTPUT:

- Here, we can see when motion detector detects the motion light is turned ON.

